2026 – 2027 | 2.1 Virtualisatie | Henk Broekman

Week 6 | Les 1

# Kubernetes



Container Orchestration
1 van 2

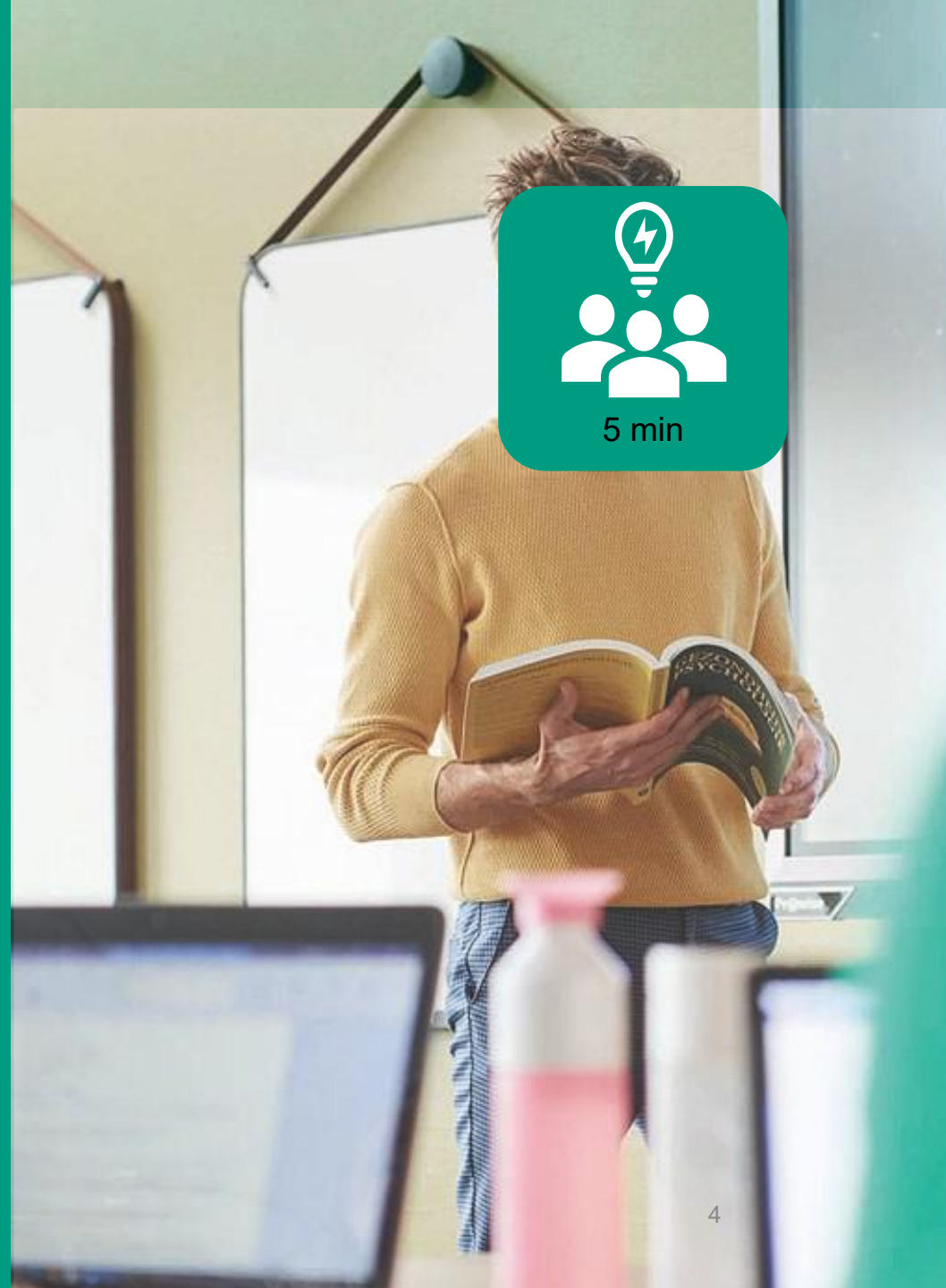
SAXION HOGESCHOOL

# Presentie

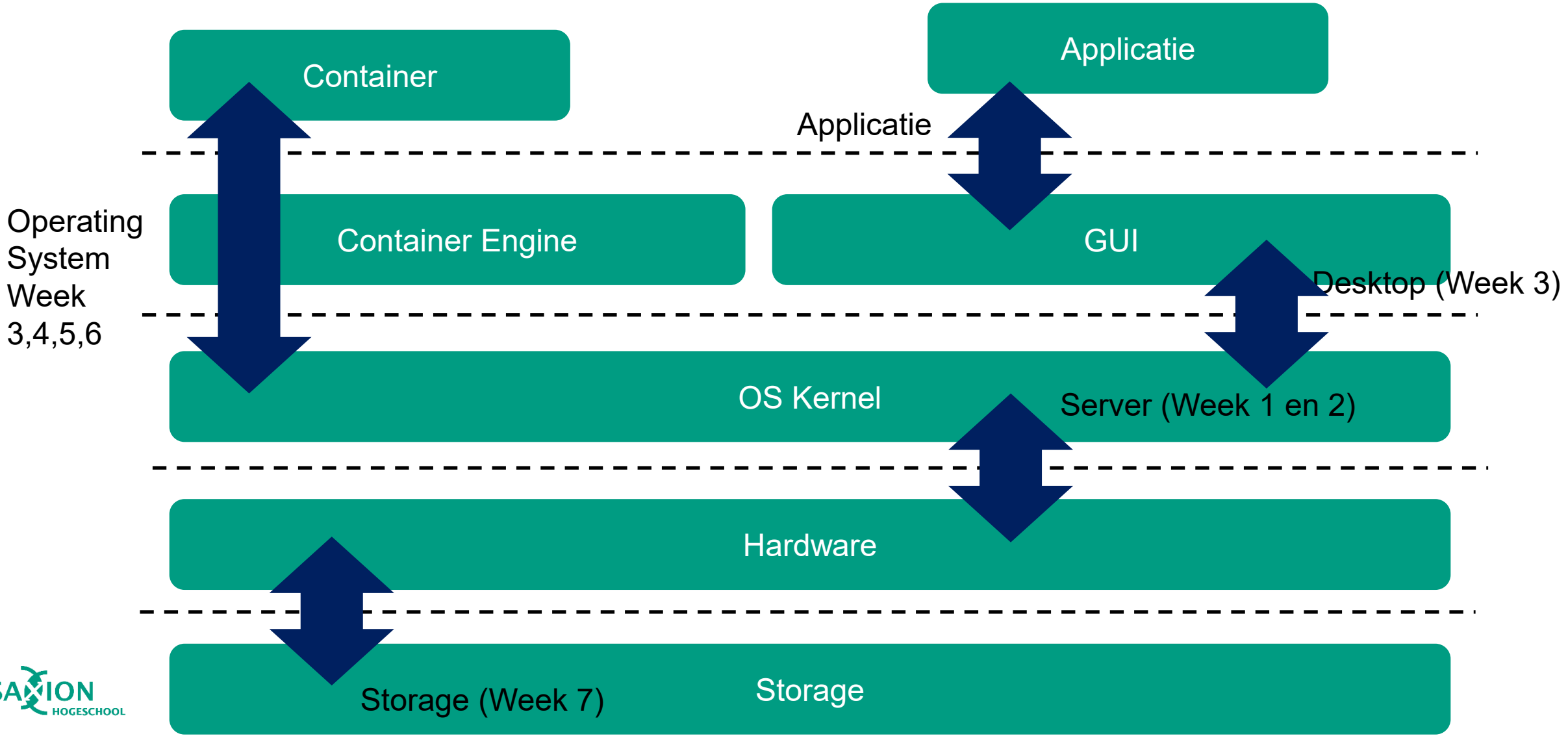


# Inventarisatie

- Wie heeft Shared Remote Virtual Desktop (SRVD) opdracht af ?
- Wie heeft VM met Docker werkend ?
- Wie heeft VM met Minikube werkend ?
- Wie heeft Docker opdracht af ?
- Wie heeft Docker Compose opdracht af ?



# Diverse vormen van virtualisatie



# Weekindeling

1. **Servervirtualisatie** (stand-alone) + Storage
2. **Servervirtualisatie** (cluster)
3. **Desktopvirtualisatie** (Remote Virtual Desktop)
4. OS/Application single **Containers** [Docker]
5. OS/Application multiple **Containers** [Docker Compose]
6. OS/Application **Container Orchestration** [Kubernetes]
7. **Object Storage**
8. Assessment

# Agenda



Kubernetes

Architectuur
Master / Control
Worker Node(s)



Kubernetes

Objecten:
Pod



Kubernetes

MiniKube

K3S
K3d

MicroK8S



KubeVirt

# Wat is Kubernetes?



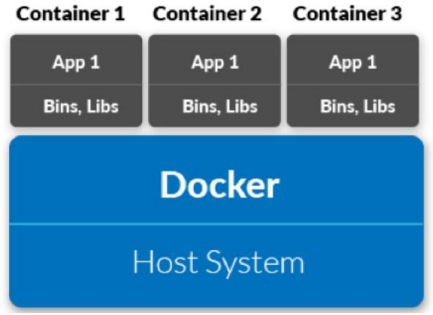
kubernetes

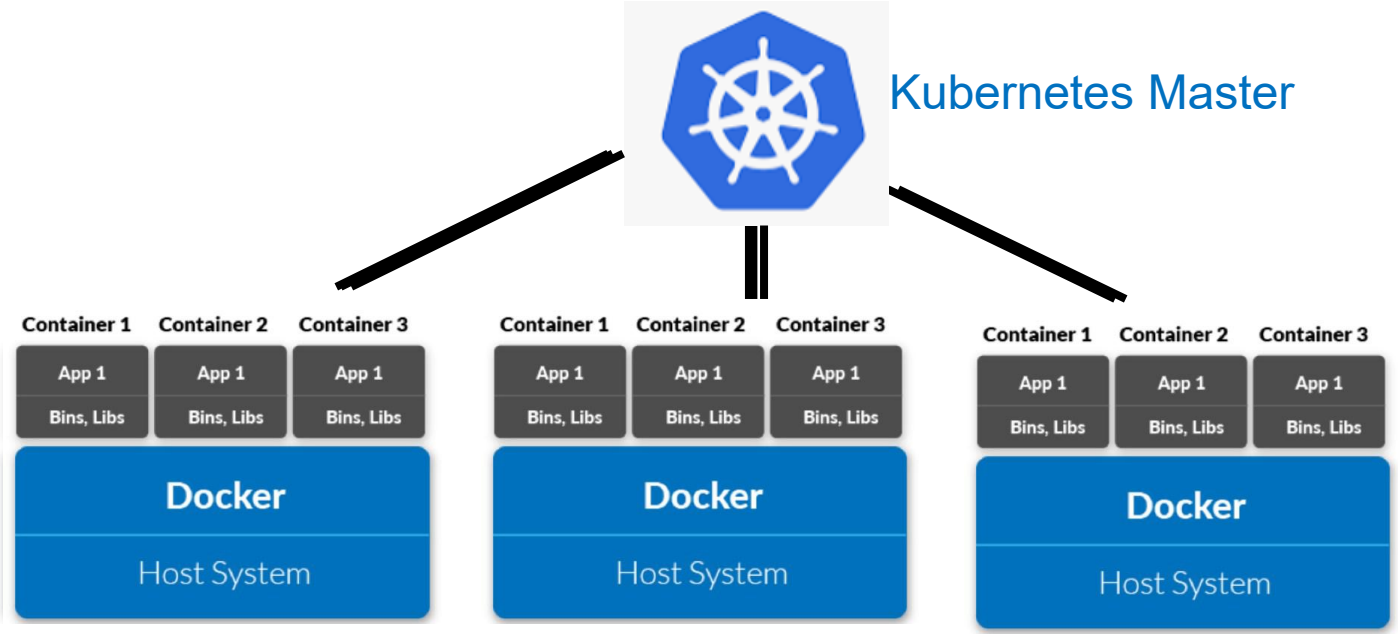
10 min.

SAXION HOGESCHOOL

# Doelstelling

- We kennen de docker host met 1 of meer containers:
- Probleem: niet schaalbaar, niet hoog beschikbaar etc.
- We willen naar situatie waar dit WEL het geval is ….




SAXION HOGESCHOOL

# Kubernetes historie

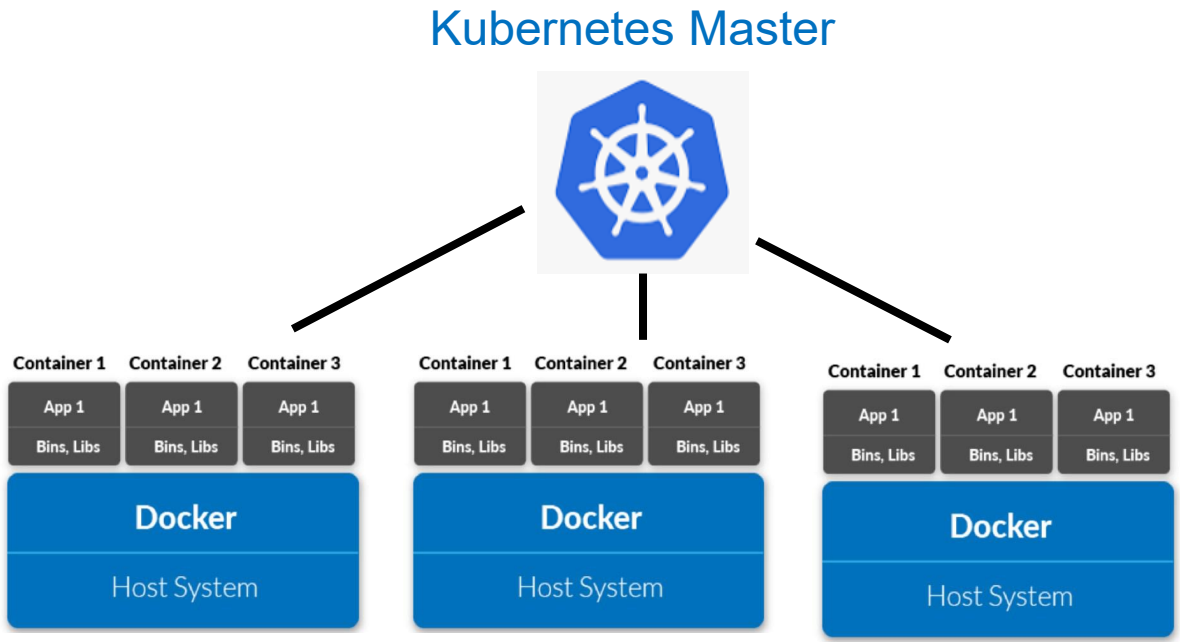
- Ontwikkeld door Google
- Grieks voor kapitein, piloot of bestuurder.
- Ook bekend als K8s (K dan 8 letters dan s)
- Versie 1 gereleased juli 2015
- Oktober 2022: versie 1.25
- Nu open source
- Alternatief Docker: Docker Swarm
- Kubernetes is Marktleider !





# Kubernetes

- Kubernetes Master: vergelijkbaar met vCenter
  - Maakt dockeromgeving hoog beschikbaar.
    - Valt container of host uit dan wordt er elders 1 herstart.
  - Schaalbaar (zorgt voor loadbalancing over containers)



# Kubernetes | Functionaliteit (wat kan het ?)

- **Service discovery en loadbalancing**
  - Applicatie kan dmv ip-adres of dns beschikbaar worden gesteld over meerdere nodes.
  - Automatische loadbalancing
- **Self healing**
  - Als een container uitvalt wordt automatisch nieuwe gestart
- **Auto scaling**
  - Als de load toeneemt kunnen automatisch pods worden bijgeschaald
- **Storage orchestration**
  - Automatisch mounten (diverse soorten) storage voor dataopslag applicatie
- **Multi Platform**
  - Nodes kunnen verdeeld zijn over verschillende cloudomgevingen pf on-premise
- Bron:
  - https://kubernetes.io/docs/concepts/overview/

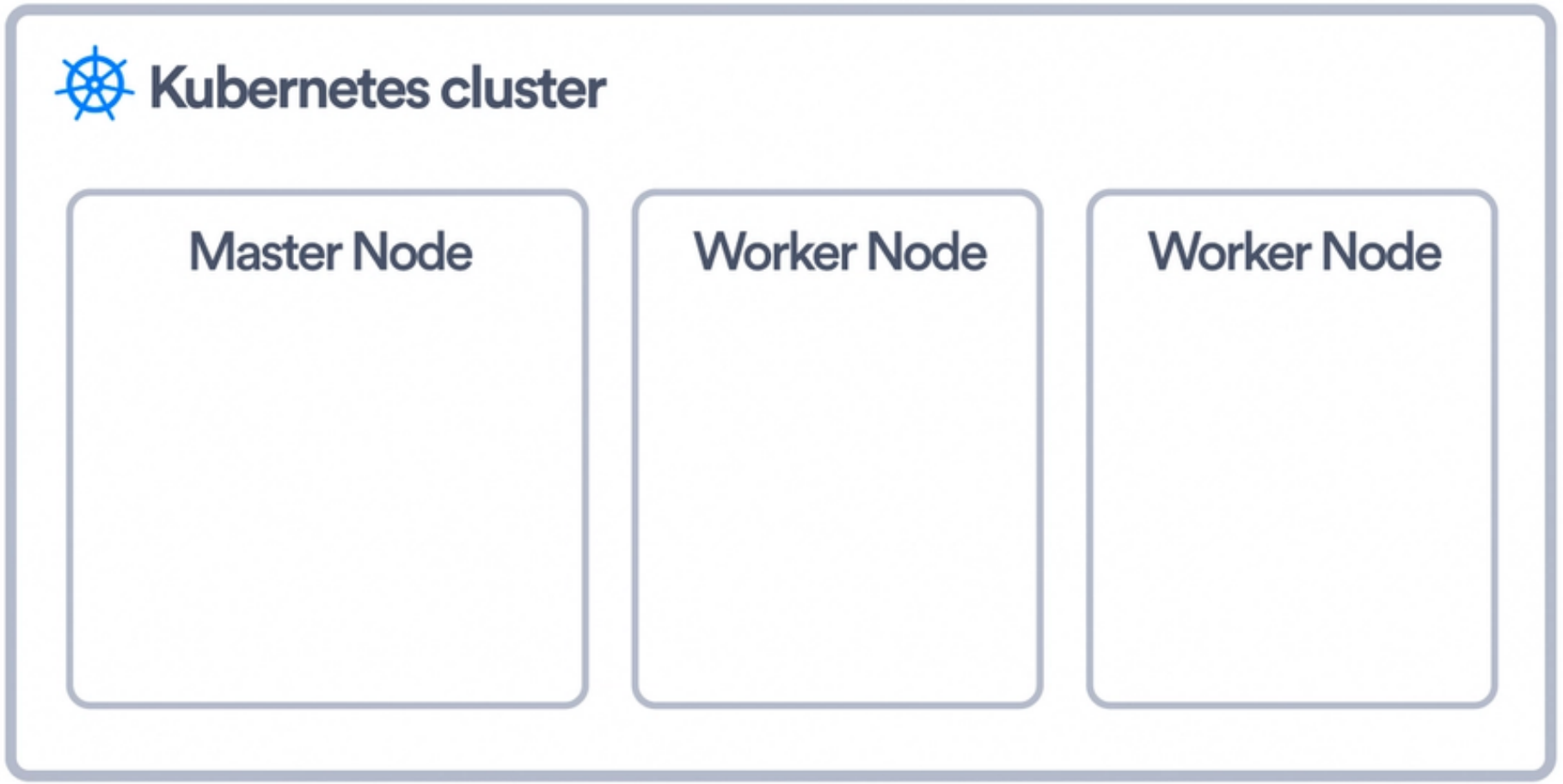
# Kubernetes | architectuur




5 min.

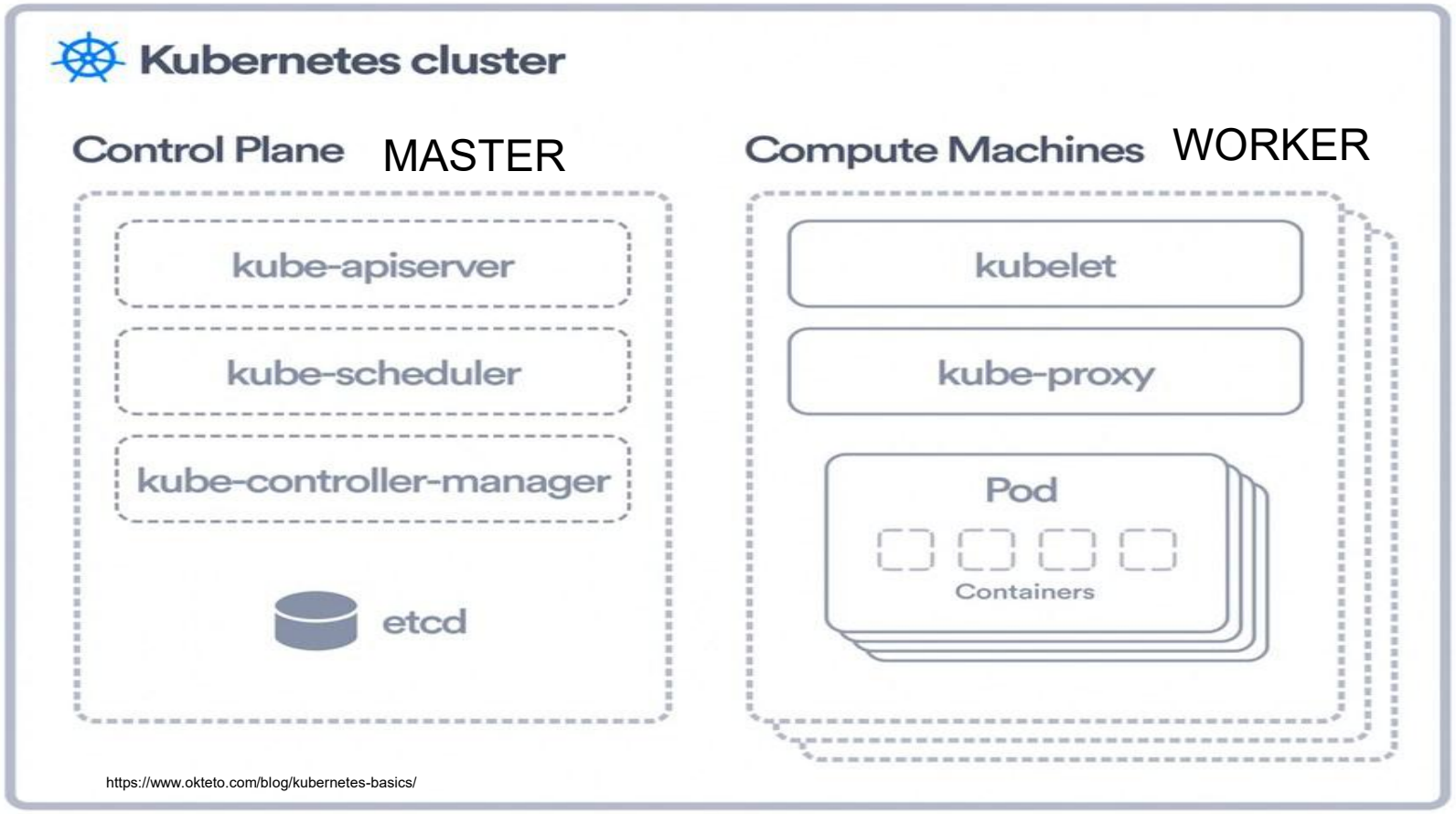
SAXION HOGESCHOOL

# Kubernetes | Architectuur




https://www.okteto.com/blog/kubernetes-basics/

# Kubernetes | Architectuur | Onderdelen

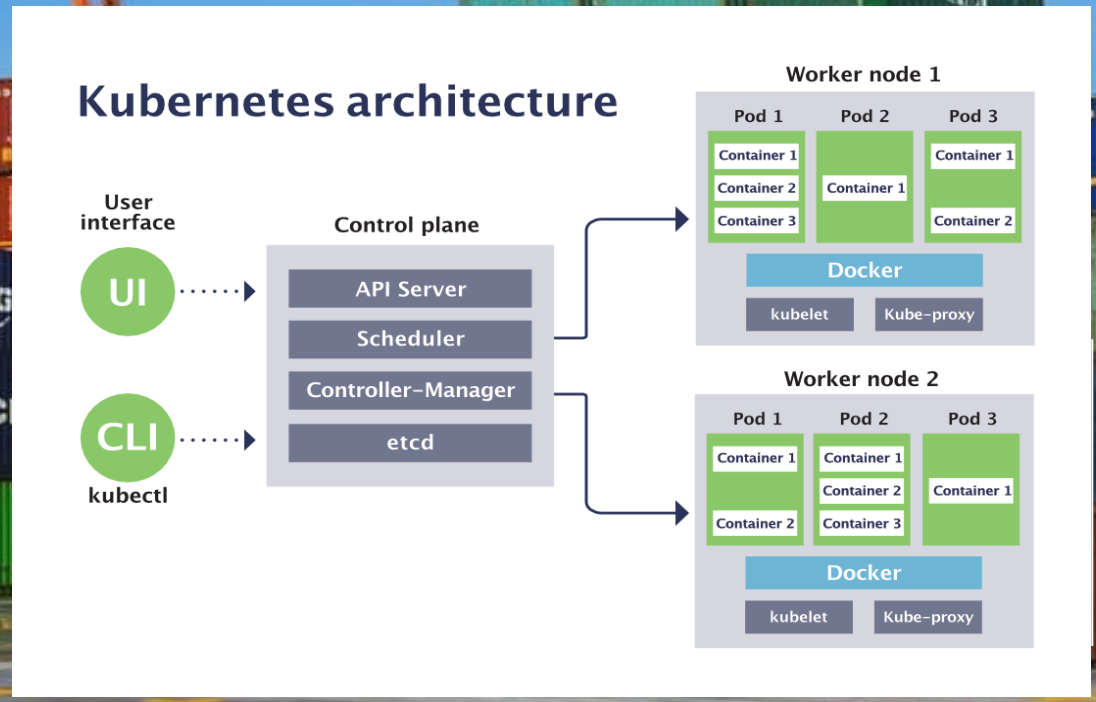



https://www.okteto.com/blog/kubernetes-basics/

https://kubernetes.io/docs/concepts/overview/components/

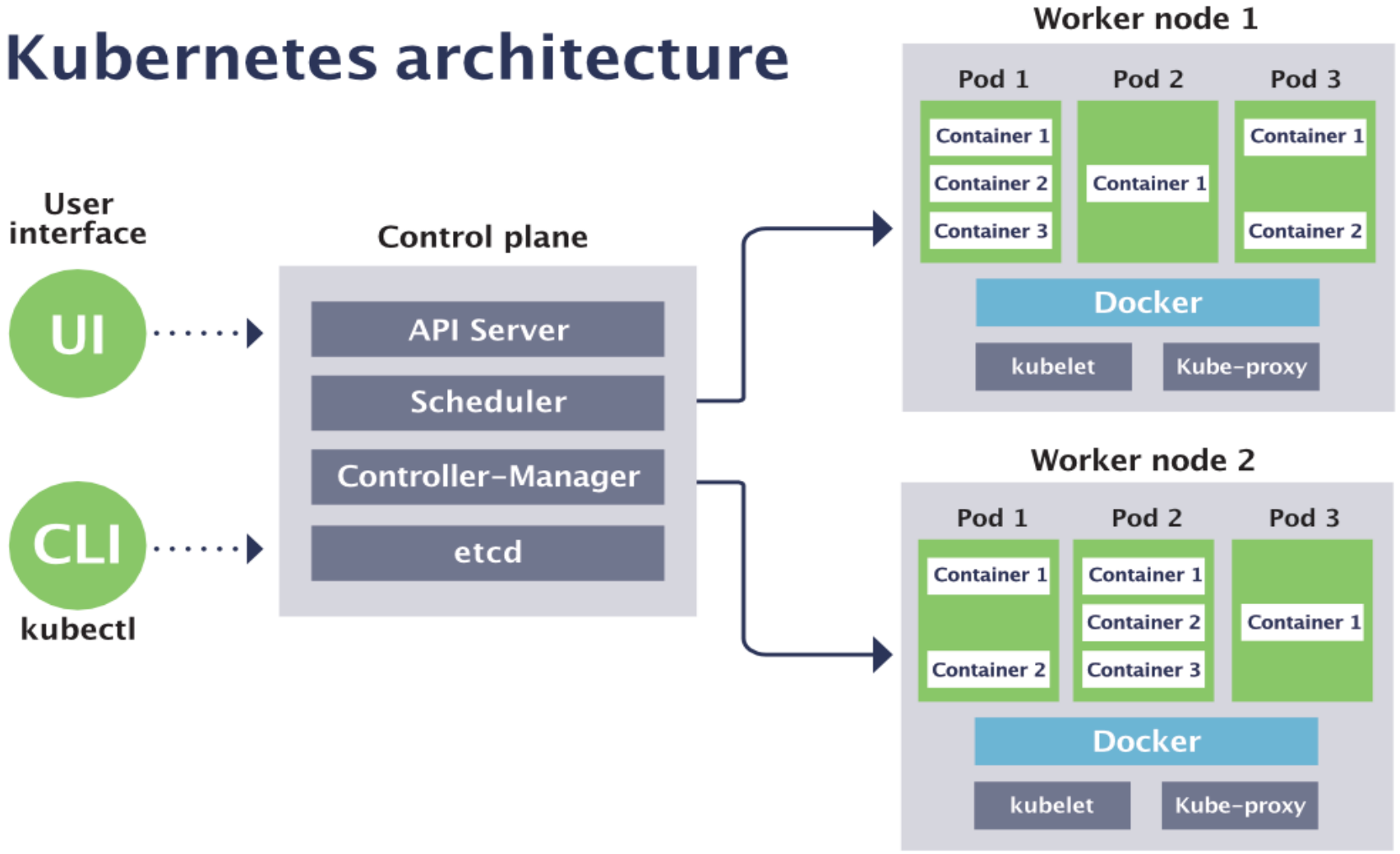
# MASTER / Control plane

12 min.




SAXION HOGESCHOOL

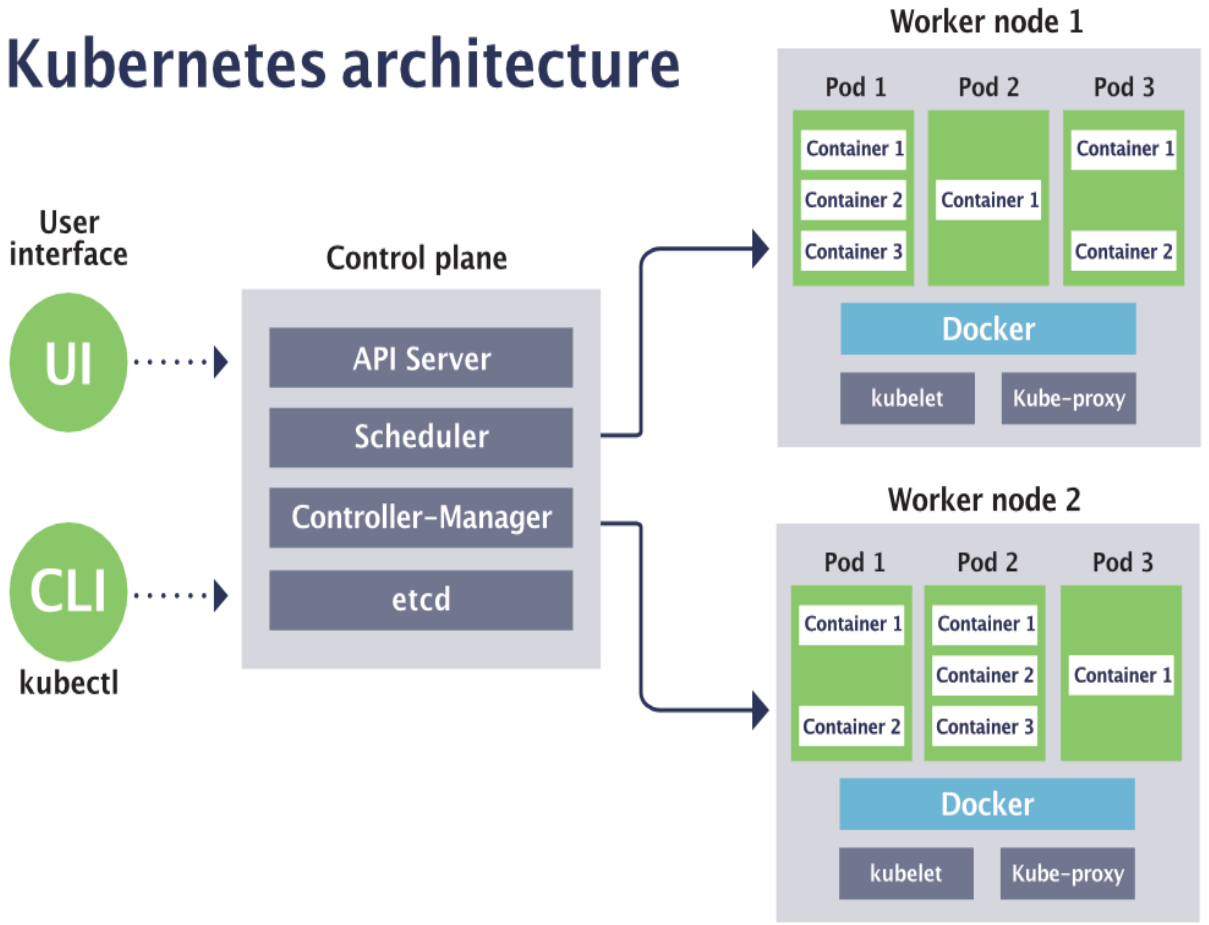
# Kubernetes | Architectuur | Details




https://images.ctfassets.net/w1bd7cq683kz/5Ex6830HzBPU5h8Ou8xQAB/2c948105fc10094348203bec6c1eab04/Kubernetes_20architecture_20diagram.png

# Kubernetes | Architectuur | Componenten | Master

- **API Server**
  - The API Server is the front-end of the control plane and the only component in the control plane that we interact with directly. Internal system components, as well as external user components, all communicate via the same API.
- **Scheduler**
  - A Scheduler watches for new requests coming from the API Server and assigns them to healthy nodes.
  - It ranks the quality of the nodes and deploys pods to the best-suited node. If there are no suitable nodes, the pods are put in a pending state until such a node appears.
- **Controller**
  - The role of the Controller is to obtain the desired state from the API Server.
  - It checks the current state of the nodes.
  - It is tasked to control, and determines if there are any differences, and resolves them, if any.
- **etcd / Key-Value Store**
  - The Key-Value Store, also called etcd, is a database Kubernetes uses to back-up all cluster data.
  - It stores the entire configuration and state of the cluster.
  - The Master node queries etcd to retrieve parameters for the state of the nodes, pods, and containers.




https://images.ctfassets.net/w1bd7cq683kz/5Ex6830HzBPU5h8Ou8xQAB/2c948105fc10094348203bec6c1eab04/Kubernetes_20architecture_20diagram.png

# Kubeadm

- Kubeadm is een tool die gebruikt wordt om Kubernetes (K8s) clusters te bouwen.
- Kubeadm voert de acties uit die nodig zijn om snel een minimaal levensvatbaar cluster op te starten.
- Door zijn ontwerp geeft het alleen om bootstrapping, niet om het leveren van machines (onderliggende worker en master nodes).
- Kubeadm dient ook als bouwsteen voor meer op maat gemaakte tooling op een hoger niveau.
- Gebruikelijke toepassingen voor Kubeadm zijn onder andere testen, het creëren van basislijnen voor meer geavanceerde K8s implementaties en het bieden van een eenvoudig startpunt voor nieuwe K8s gebruikers voor de configuratie van clusters.

- Bron: https://www.densify.com/kubernetes-tools/kubeadm/

# kubeadm init

This command initializes a Kubernetes control-plane node.

Run this command in order to set up the Kubernetes control plane

## Synopsis

Run this command in order to set up the Kubernetes control plane

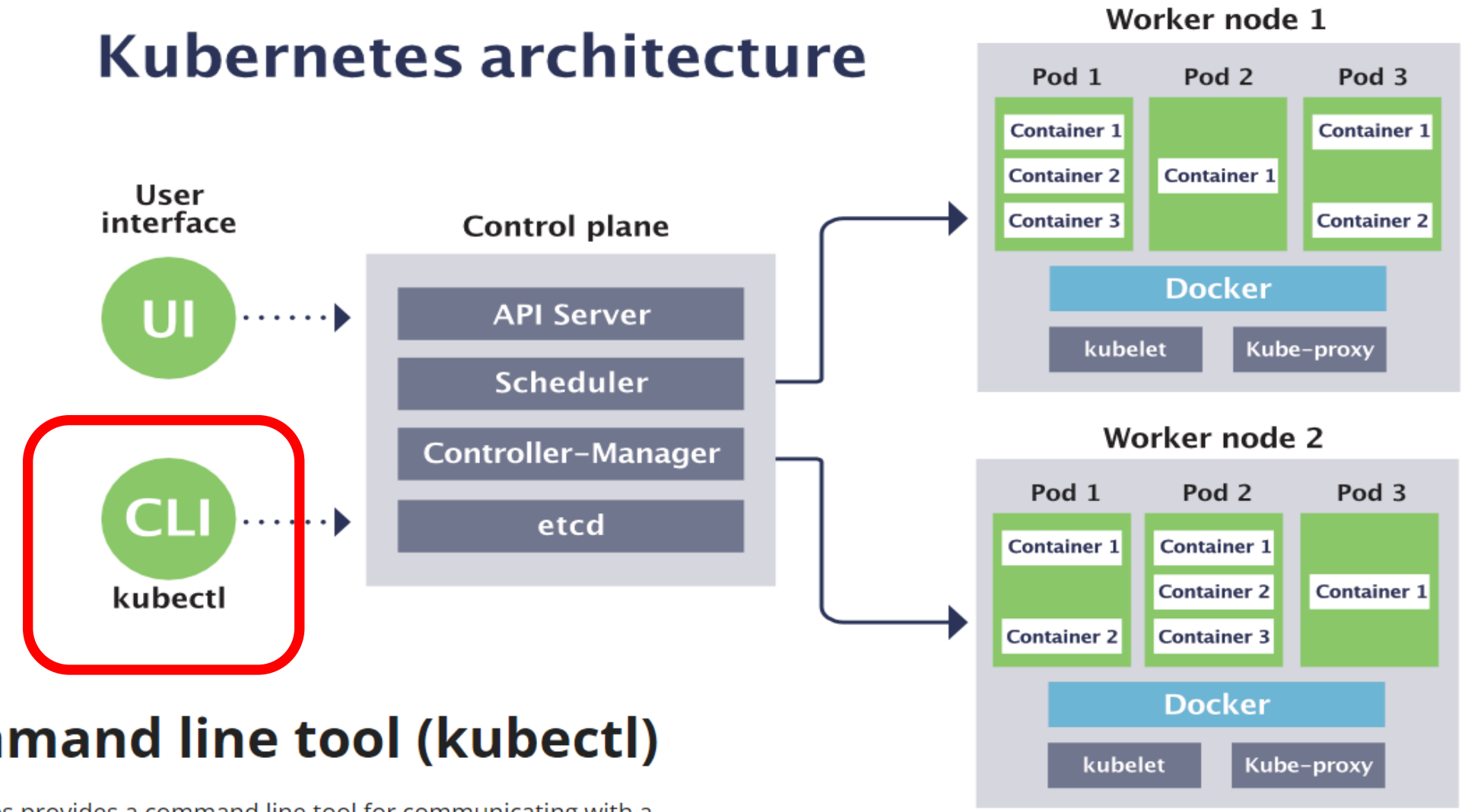
The "init" command executes the following phases:

```
preflight                     Run pre-flight checks
certs                         Certificate generation
  /ca                           Generate the self-signed Kubernetes CA to provision identities for other Kubernetes com
  /apiserver                    Generate the certificate for serving the Kubernetes API
  /apiserver-kubelet-client     Generate the certificate for the API server to connect to kubelet
  /front-proxy-ca               Generate the self-signed CA to provision identities for front proxy
  /front-proxy-client           Generate the certificate for the front proxy client
  /etcd-ca                      Generate the self-signed CA to provision identities for etcd
  /etcd-server                  Generate the certificate for serving etcd
  /etcd-peer                    Generate the certificate for etcd nodes to communicate with each other
  /etcd-healthcheck-client      Generate the certificate for liveness probes to healthcheck etcd
  /apiserver-etcd-client        Generate the certificate the apiserver uses to access etcd
  /sa                           Generate a private key for signing service account tokens along with its public key
```



https://kubernetes.io/docs/reference/setup-tools/kubeadm/kubeadm-init/

# Kubectl




## Command line tool (kubectl)

Kubernetes provides a command line tool for communicating with a Kubernetes cluster's control plane, using the Kubernetes API.

SAXION HOGESCHOOL

# Kubectl | Syntax

```
kubectl [command] [TYPE] [NAME] -o <output_format>
```

Depending on the `kubectl` operation, the following output formats are supported:

| Output format | Description |
| --- | --- |
| `-o custom-columns=<spec>` | Print a table using a comma separated list of custom columns. |
| `-o custom-columns-file=<filename>` | Print a table using the custom columns template in the `<filename>` file. |
| `-o json` | Output a JSON formatted API object. |
| `-o jsonpath=<template>` | Print the fields defined in a jsonpath expression. |
| `-o jsonpath-file=<filename>` | Print the fields defined by the jsonpath expression in the `<filename>` file. |
| `-o name` | Print only the resource name and nothing else. |
| `-o wide` | Output in the plain-text format with any additional information. For pods, the node name is included. |
| `-o yaml` | Output a YAML formatted API object. |

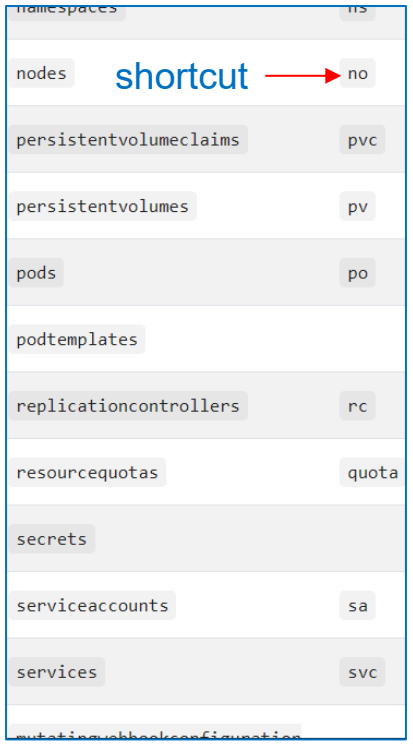
SAXION HOGESCHOOL

# Kubectl | syntax | command, type & name

Zie https://kubernetes.io/docs/reference/kubectl/overview/

```
kubectl [command] [TYPE] [NAME] [flags]
```

get
apply
create
delete
describe
run
version
attach
autoscale
…




pods
pod1
services
deployments
…

Help:
kubectl <command> --help

- **Manifest**
  Specification of a Kubernetes API object in JSON or YAML format.[-]
  A manifest specifies the desired state of an object that Kubernetes will maintain when you apply the manifest. Each configuration file can contain multiple manifests.

# Creating objects

Kubernetes manifests can be defined in YAML or JSON. The file extension `.yaml`, `.yml`, and `.json` can be used.

```
kubectl apply -f ./my-manifest.yaml            # create resource(s)
kubectl apply -f ./my1.yaml -f ./my2.yaml      # create from multiple files
kubectl apply -f ./dir                         # create resource(s) in all manifest files in dir
kubectl apply -f https://git.io/vPieo          # create resource(s) from url
kubectl create deployment nginx --image=nginx  # start a single instance of nginx

# create a Job which prints "Hello World"
kubectl create job hello --image=busybox:1.28 -- echo "Hello World"

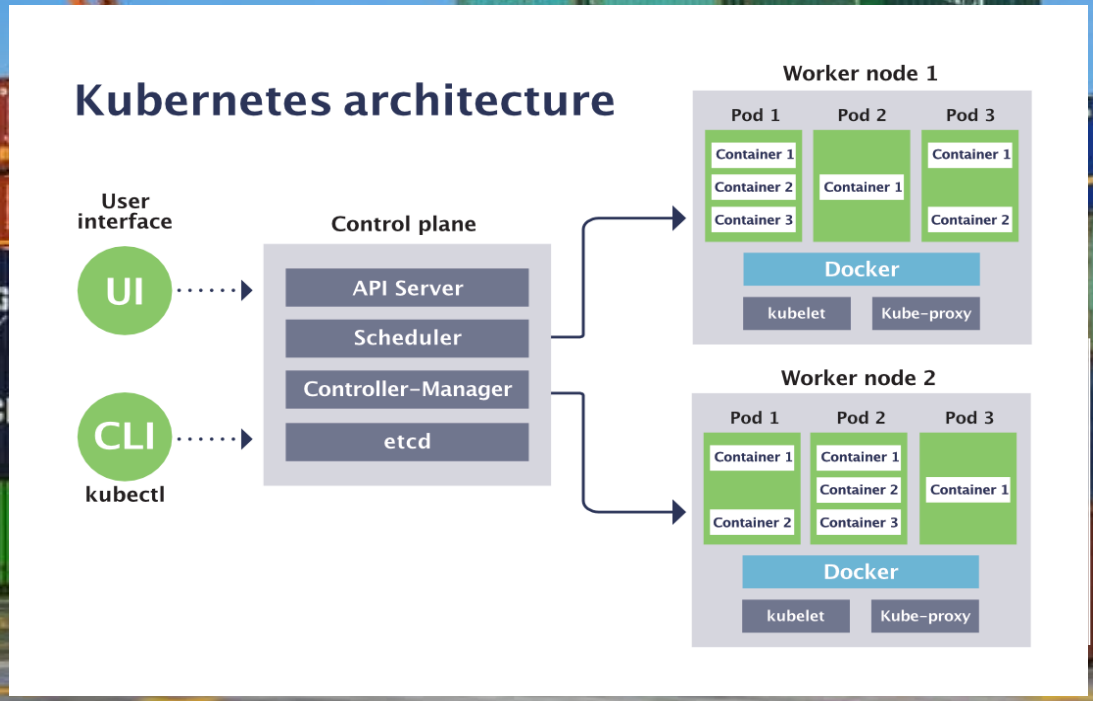
# create a CronJob that prints "Hello World" every minute
kubectl create cronjob hello --image=busybox:1.28   --schedule="*/1 * * * *" -- echo "Hello World"

kubectl explain pods                           # get the documentation for pod manifests
```

https://kubernetes.io/docs/reference/kubectl/cheatsheet/

# Worker node(s)

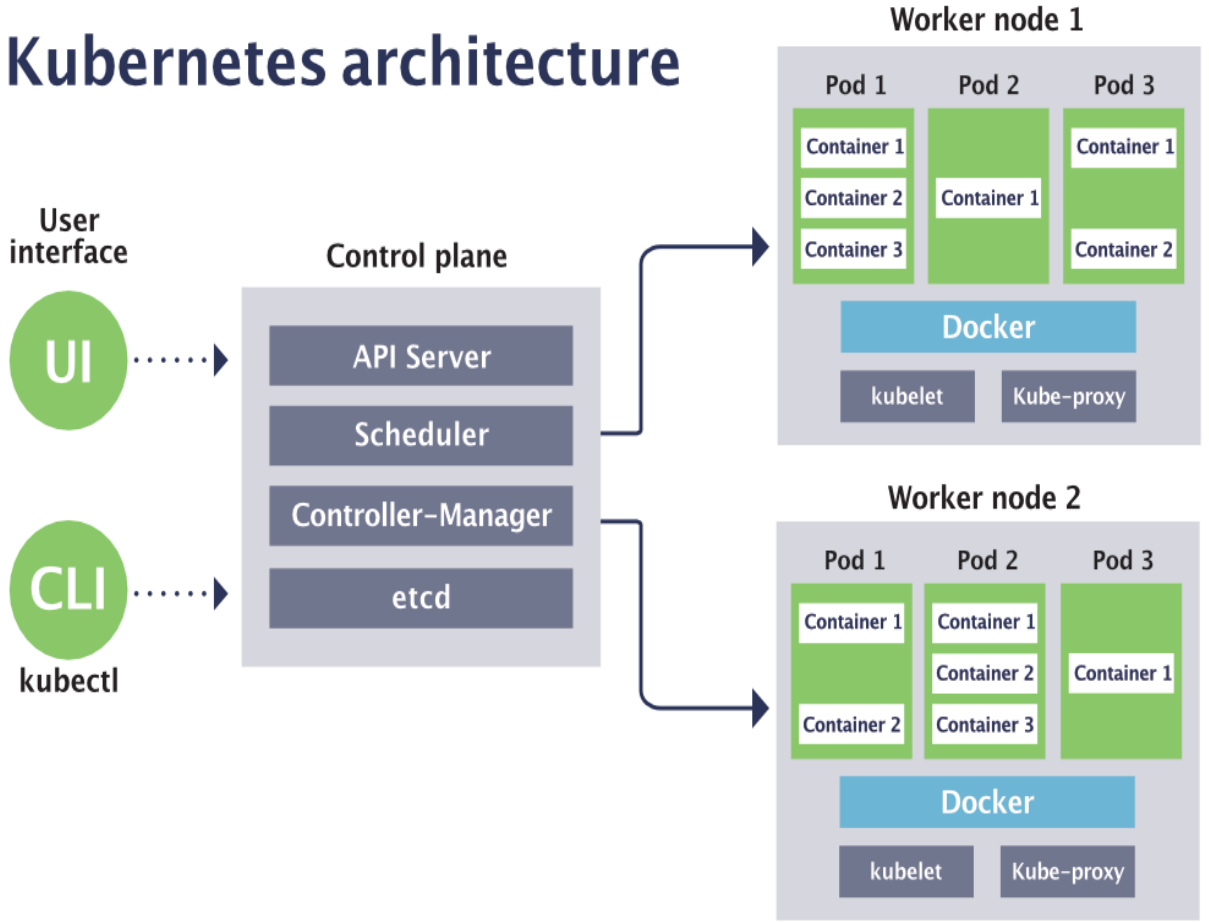
4 min.




SAXION HOGESCHOOL

# Kubernetes | Architectuur | Componenten | (Worker) Node

- **Kubelet**
  - The kubelet runs on every node in the cluster. It is the principal Kubernetes agent.
    - By installing kubelet, the node's CPU, RAM, and storage become part of the broader cluster. It watches for tasks sent from the API Server, executes the task, and reports back to the Master.
    - It also monitors pods and reports back to the control panel if a pod is not fully functional. Based on that information, the Master can then decide how to allocate tasks and resources to reach the desired state.
- **Container Runtime**
  - The container runtime pulls images from a container image registry and starts and stops containers. A 3rd party software or plugin, such as Docker, usually performs this function.
- **Kube-proxy**
  - The kube-proxy makes sure that each node gets its IP address, implements local iptables and rules to handle routing and traffic load-balancing.



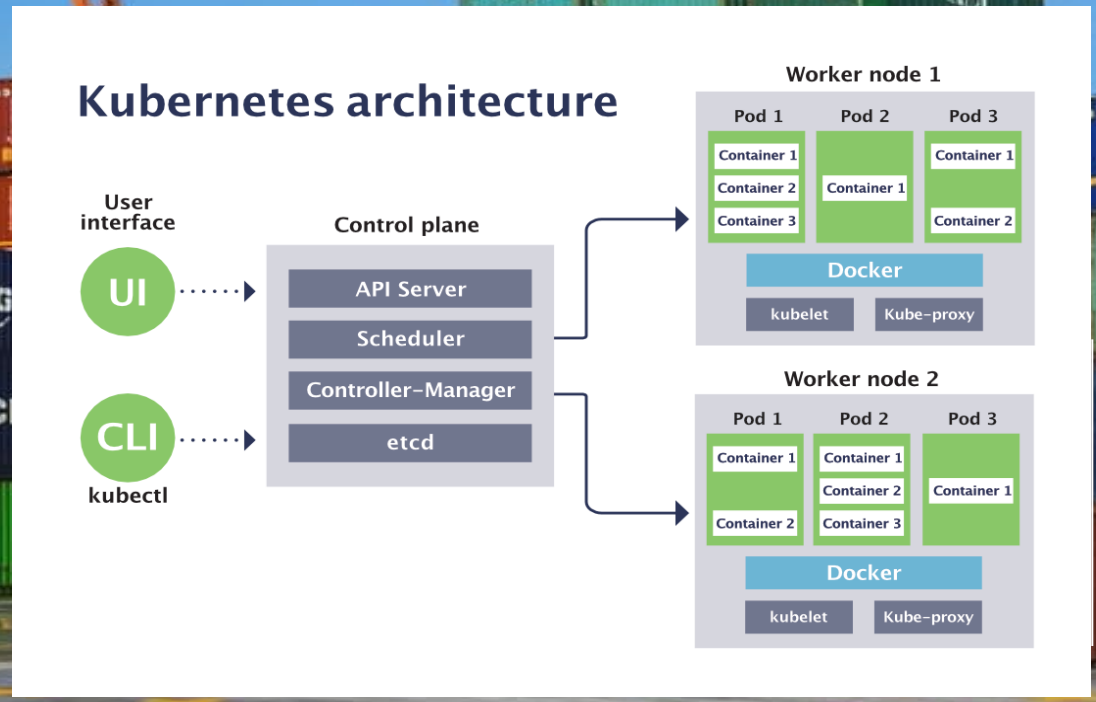

https://images.ctfassets.net/w1bd7cq683kz/5Ex6830HzBPU5h8Ou8xQAB/2c948105fc10094348203bec6c1eab04/Kubernetes_20architecture_20diagram.png

# Kubernetes | Key Components

- POD
- Deployment
  - A deployment is responsible for keeping a set of pods running.
- Service
  - A service is responsible for enabling network access to a set of pods.

# Kubernetes Objecten

15 min.



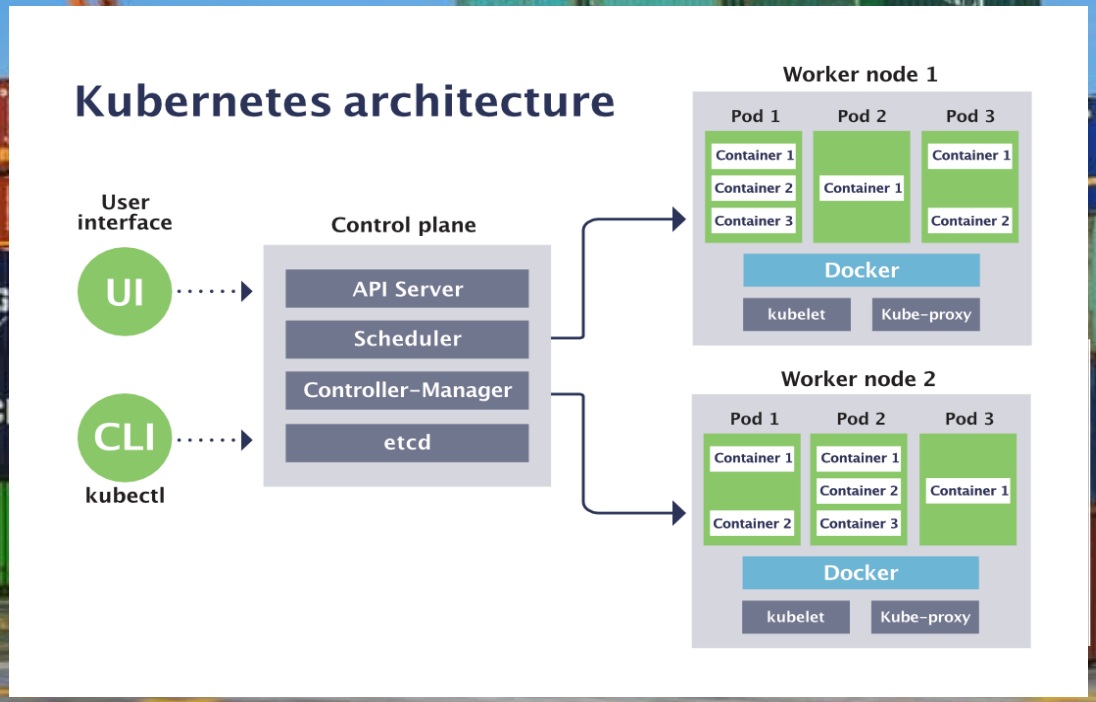

SAXION HOGESCHOOL

# Kubernetes Objecten

- **Workload: Pod**
  - Een pod is de *kleinste en eenvoudigste eenheid* in de Kubernetes-objectenhiërarchie.
  - Het is een enkele instantie van een applicatie in Kubernetes, die één of meer containers kan bevatten.
  - Een pod kan worden gebruikt om één of meer containers te starten en te beheren.
- **Workload: ReplicaSet**
  - Een ReplicaSet is een Kubernetes-object dat ervoor zorgt dat een bepaald aantal pod-replica's op elk moment wordt uitgevoerd.
  - Het is echter een hoger niveau concept dan een Pod en biedt declaratieve updates aan Pods samen met veel andere nuttige functies.
- **Workload: Deployment**
  - Een Deployment is een *hoger niveau concept dan ReplicaSets* en biedt declaratieve updates aan Pods samen met veel andere nuttige functies.
  - Het beheert ReplicaSets en zorgt voor declaratieve updates aan Pods.
- **Service:**
  - Een Service is een abstractie die definieert hoe Pods toegankelijk zijn binnen het cluster.
  - Het biedt een stabiel IP-adres en DNS-naam voor de set Pods die het vertegenwoordigt.

https://kubernetes.io/docs/concepts/workloads/
https://kubernetes.io/docs/concepts/services-networking/service/

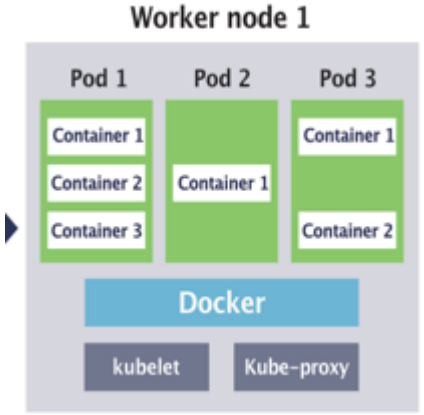
# Kubernetes Objecten | POD




SAXION HOGESCHOOL

# Kubernetes | Architectuur | Componenten | Node | POD

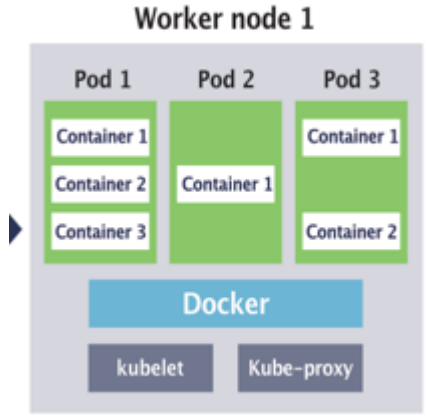
- Een Pod is de kleinste, atomaire deployment unit in Kubernetes.
  - Het is de kleinste eenheid die je kunt maken, implementeren en beheren.
- Een Pod kan één of (meestal) meerdere containers bevatten die een gedeelde levenscyclus hebben.
  - Deze containers delen resources en netwerk.
- Alle containers binnen één Pod delen dezelfde netwerk namespace.
  - Ze hebben hetzelfde IP-adres en poorten, en kunnen elkaar bereiken via LOCALHOST.
- Containers binnen een Pod kunnen één of meer volumes delen.
  - Dit maakt het mogelijk voor containers om bestanden met elkaar uit te wisselen of om persistente data te gebruiken.




Bron:
Google Gemini

# Kubernetes | Architectuur | Componenten | Node | POD

- Pods zijn ontworpen om van nature vluchtig te zijn (ephemeral).
  - Als een Pod om welke reden dan ook sterft, wordt deze niet automatisch herstart; een controller creëert een nieuwe.
- De levenscyclus van een Pod wordt beheerd door controllers zoals Deployments, StatefulSets of ReplicaSets.
  - Deze controllers zorgen ervoor dat het gewenste aantal Pods draait.
- Elke Pod krijgt een uniek IP-adres toegewezen binnen het Kubernetes-cluster.




Bron:
Google Gemini

# Voorbeeld




MariaDB 12.0 en Wordpress 6.8
Samen 1 CMS omgeving

# Viewing and finding resources

https://kubernetes.io/docs/reference/kubectl/cheatsheet/

```
# Get commands with basic output
kubectl get services                          # List all services in the namespace
kubectl get pods --all-namespaces             # List all pods in all namespaces
kubectl get pods -o wide                      # List all pods in the current namespace, with more details
kubectl get deployment my-dep                 # List a particular deployment
kubectl get pods                              # List all pods in the namespace
kubectl get pod my-pod -o yaml                # Get a pod's YAML

# Describe commands with verbose output
kubectl describe nodes my-node
kubectl describe pods my-pod

# List Services Sorted by Name
kubectl get services --sort-by=.metadata.name

# List pods Sorted by Restart Count
kubectl get pods --sort-by='.status.containerStatuses[0].restartCount'

# List PersistentVolumes sorted by capacity
kubectl get pv --sort-by=.spec.capacity.storage

# Get the version label of all pods with label app=cassandra
kubectl get pods --selector=app=cassandra -o \
  jsonpath='{.items[*].metadata.labels.version}'

# Retrieve the value of a key with dots, e.g. 'ca.crt'
kubectl get configmap myconfig \
  -o jsonpath='{.data.ca\.crt}'

# Retrieve a base64 encoded value with dashes instead of underscores.
kubectl get secret my-secret --template='{{index .data "key-name-with-dashes"}}'
```

kubectl get pods

S

# Interacting with running Pods

```
kubectl logs my-pod                                 # dump pod logs (stdout)
kubectl logs -l name=myLabel                        # dump pod logs, with label name=myLabel (stdout)
kubectl logs my-pod --previous                      # dump pod logs (stdout) for a previous instantiation of a container
kubectl logs my-pod -c my-container                 # dump pod container logs (stdout, multi-container case)
kubectl logs -l name=myLabel -c my-container        # dump pod logs, with label name=myLabel (stdout)
kubectl logs my-pod -c my-container --previous      # dump pod container logs (stdout, multi-container case) for a previous instantiation of a conta
kubectl logs -f my-pod                              # stream pod logs (stdout)
kubectl logs -f my-pod -c my-container              # stream pod container logs (stdout, multi-container case)
kubectl logs -f -l name=myLabel --all-containers    # stream all pods logs with label name=myLabel (stdout)
kubectl run -i --tty busybox --image=busybox:1.28 -- sh  # Run pod as interactive shell
kubectl run nginx --image=nginx -n mynamespace      # Start a single instance of nginx pod in the namespace of mynamespace
kubectl run nginx --image=nginx --dry-run=client -o yaml > pod.yaml
                                                    # Generate spec for running pod nginx and write it into a file called pod.yaml
kubectl attach my-pod -i                            # Attach to Running Container
kubectl port-forward my-pod 5000:6000               # Listen on port 5000 on the local machine and forward to port 6000 on my-pod
kubectl exec my-pod -- ls /                         # Run command in existing pod (1 container case)
kubectl exec --stdin --tty my-pod -- /bin/sh        # Interactive shell access to a running pod (1 container case)
kubectl exec my-pod -c my-container -- ls /         # Run command in existing pod (multi-container case)
kubectl top pod POD_NAME --containers               # Show metrics for a given pod and its containers
kubectl top pod POD_NAME --sort-by=cpu              # Show metrics for a given pod and sort it by 'cpu' or 'memory'
```

https://kubernetes.io/docs/reference/kubectl/cheatsheet/

SAXION HOGESCHOOL

# Kubernetes Objecten

- **ConfigMap:**
  - Een ConfigMap is een Kubernetes-object dat configuratiegegevens opslaat, zoals omgevingsvariabelen, bestandsinhoud of commando's.
  - ConfigMaps kunnen worden gebruikt om configuratie-instellingen van een applicatie te scheiden van de containerimages, waardoor configuratiebeheer flexibeler en gemakkelijker wordt.
- **Secret:**
  - Een Secret is een Kubernetes-object dat vertrouwelijke informatie, zoals wachtwoorden, tokens of certificaten, veilig opslaat.
  - Secrets worden gebruikt om gevoelige gegevens te beschermen en veilig beschikbaar te stellen aan applicaties binnen het cluster, zonder dat deze expliciet in de configuratiebestanden worden vermeld.

https://kubernetes.io/docs/concepts/configuration/configmap/
https://kubernetes.io/docs/concepts/configuration/secret/

# We stoppen voor nu even met Kubernetes Objecten

- Les 2 gaan we verder met:
  - Workload / Replicaset
  - Workload / Deployment
  - Services


P4 A1 U1 S1 E1
10 min

# Kubernetes Alternatieven



# Kubernetes Alternatieven

- Minikube
- K3S
- K3d
- K8S
- KIND (Kubernetes in Docker)

# Kubernetes Alternatieven



minikube

SAXION HOGESCHOOL

# Wat is MiniKube

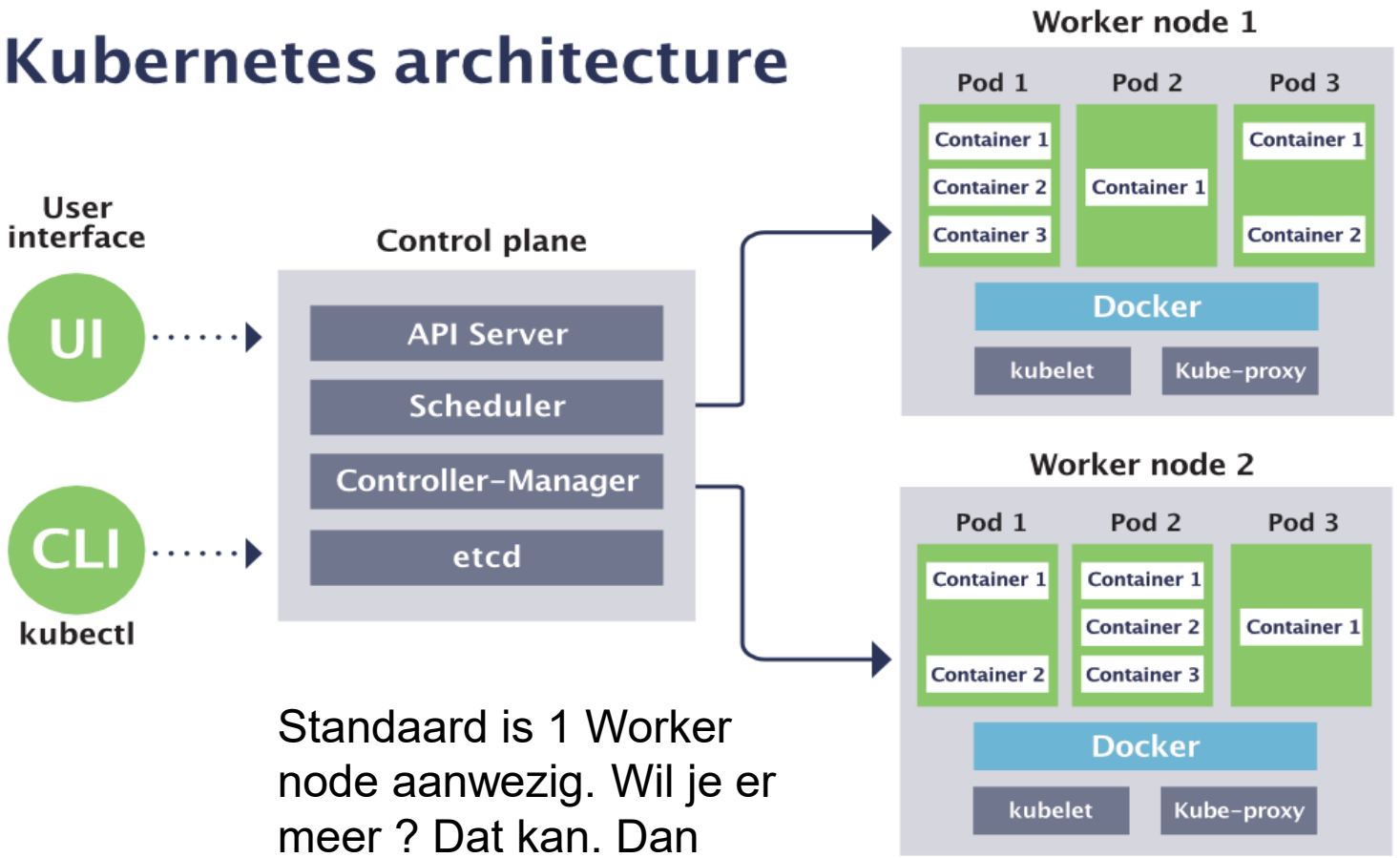
- Lightweight Kubernetes installatie op een single host machine
  - Beschikbaar voor Windows, Linux en Mac OS
  - **Master** componenten draaien op host
  - **Worker** node componenten draaien op virtuele machine op host
  - Maakt gebruik van een bestaande container runtime
    - (is niet zelf een container runtime)
    - Docker, containerd, etc.
  - Bedoeld voor ontwikkelen en testen van applicaties in Kubenetes
    - Gebruikt dezelfde YAML configuratiebestanden
    - NIET bedoeld voor productieomgevingen
      - Draait op 1 host (fysiek of virtueel)
      - Dus geen high availability
      - Dus geen schaalbaarheid



minikube

# Minikube | Architectuur | Details

Control plane
Draait rechtstreeks op het aanwezige operating systeem (o.a. Windows en Linux)
op laptop of PC




Standaard is 1 Worker node aanwezig. Wil je er meer ? Dat kan. Dan meerdere VM's OF Nested Dockers

Worker node is een virtuele machine bovenop een hypervisor (bijv. VMWare Workstation Pro of Virtualbox)

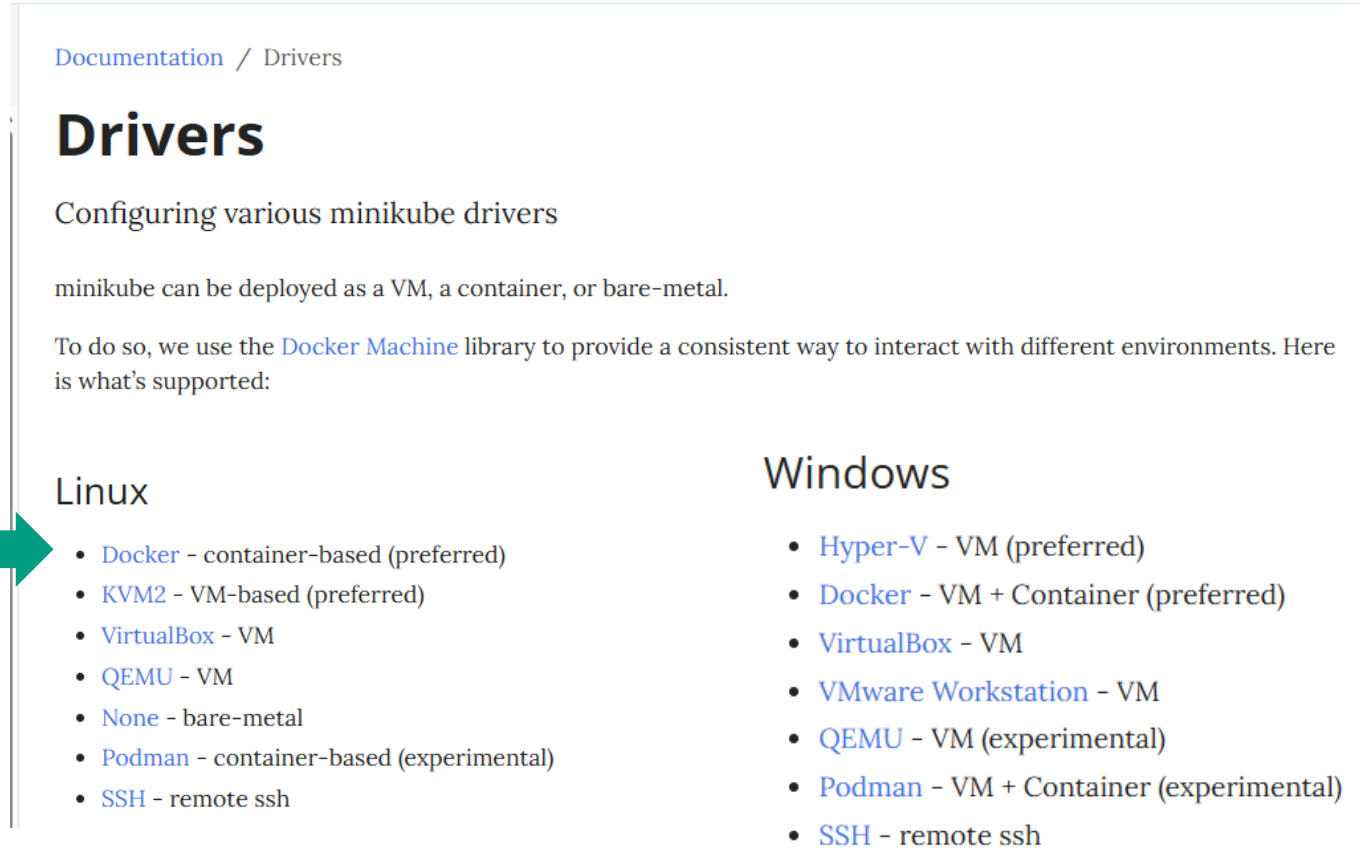
OF

Docker met Containers bovenop Docker (nested docker)

https://images.ctfassets.net/w1bd7cq683kz/5Ex6830HzBPU5h8Ou8xQAB/2c948105fc10094348203bec6c1eab04/Kubernetes_20architecture_20diagram.png

SAXION HOGESCHOOL

# Minikube | Drivers voor Worker Node(s)



Documentation / Drivers

## Drivers

Configuring various minikube drivers

minikube can be deployed as a VM, a container, or bare-metal.

To do so, we use the Docker Machine library to provide a consistent way to interact with different environments. Here is what's supported:

### Linux

- Docker - container-based (preferred)
- KVM2 - VM-based (preferred)
- VirtualBox - VM
- QEMU - VM
- None - bare-metal
- Podman - container-based (experimental)
- SSH - remote ssh

### Windows

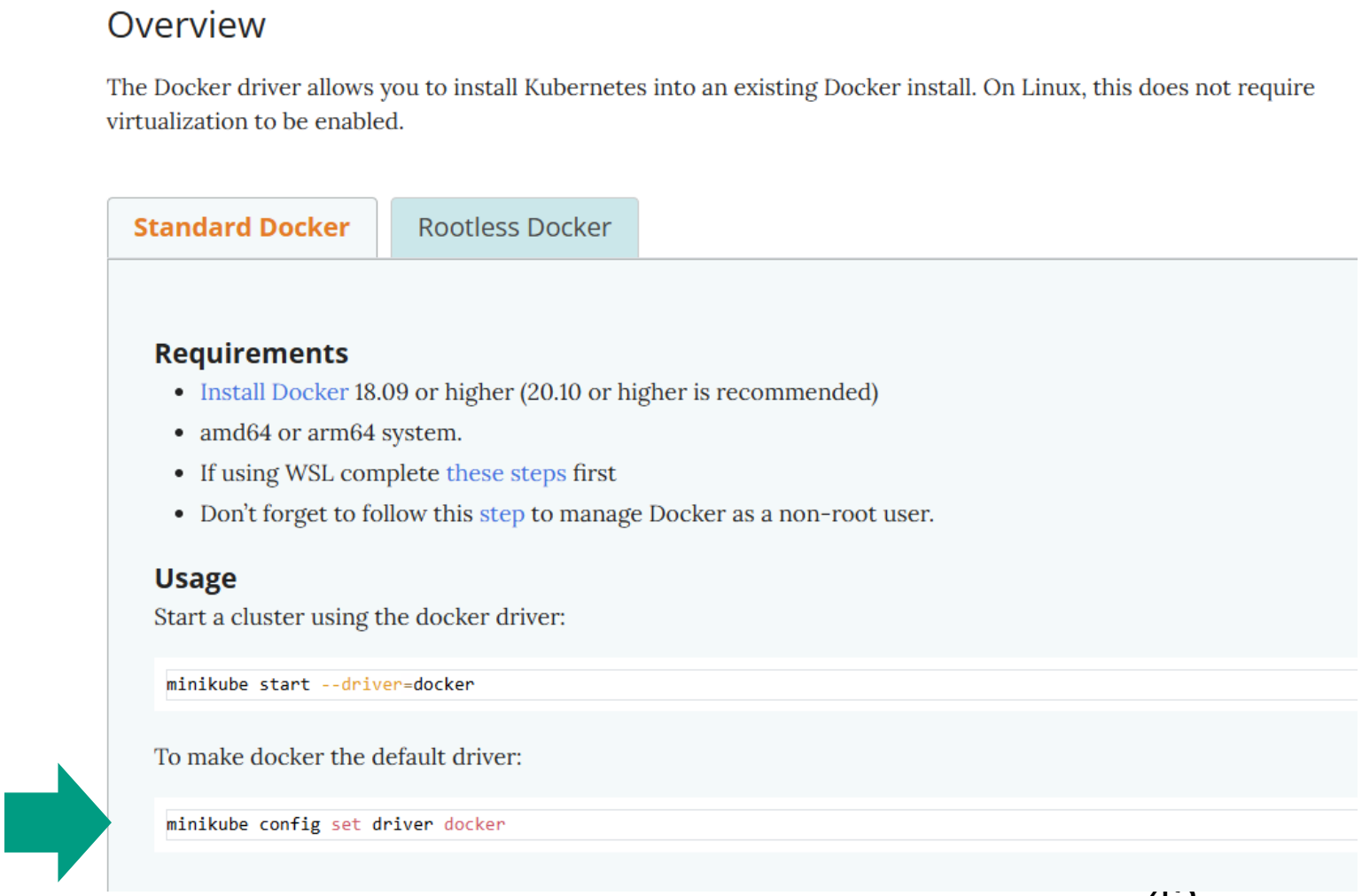
- Hyper-V - VM (preferred)
- Docker - VM + Container (preferred)
- VirtualBox - VM
- VMware Workstation - VM
- QEMU - VM (experimental)
- Podman - VM + Container (experimental)
- SSH - remote ssh

Binnen de virtuele machine (Ubuntu 24.04 LTS Linux) met Minikube wordt Docker gebruikt als Driver.

# Minikube | Driver | Docker | Instellen

Na Minikube delete worden alle instellingen verwijderd, dus ook de Driver instelling.

TIP!
Geef elke keer minikube config set driver docker voor elke minikube start.



## Overview

The Docker driver allows you to install Kubernetes into an existing Docker install. On Linux, this does not require virtualization to be enabled.

Standard Docker | Rootless Docker

### Requirements

- Install Docker 18.09 or higher (20.10 or higher is recommended)
- amd64 or arm64 system.
- If using WSL complete these steps first
- Don't forget to follow this step to manage Docker as a non-root user.

### Usage

Start a cluster using the docker driver:

```
minikube start --driver=docker
```

To make docker the default driver:

```
minikube config set driver docker
```

# Minikube | Starten omgeving

- Minikube starten met Master Node en 1 Worker Node:

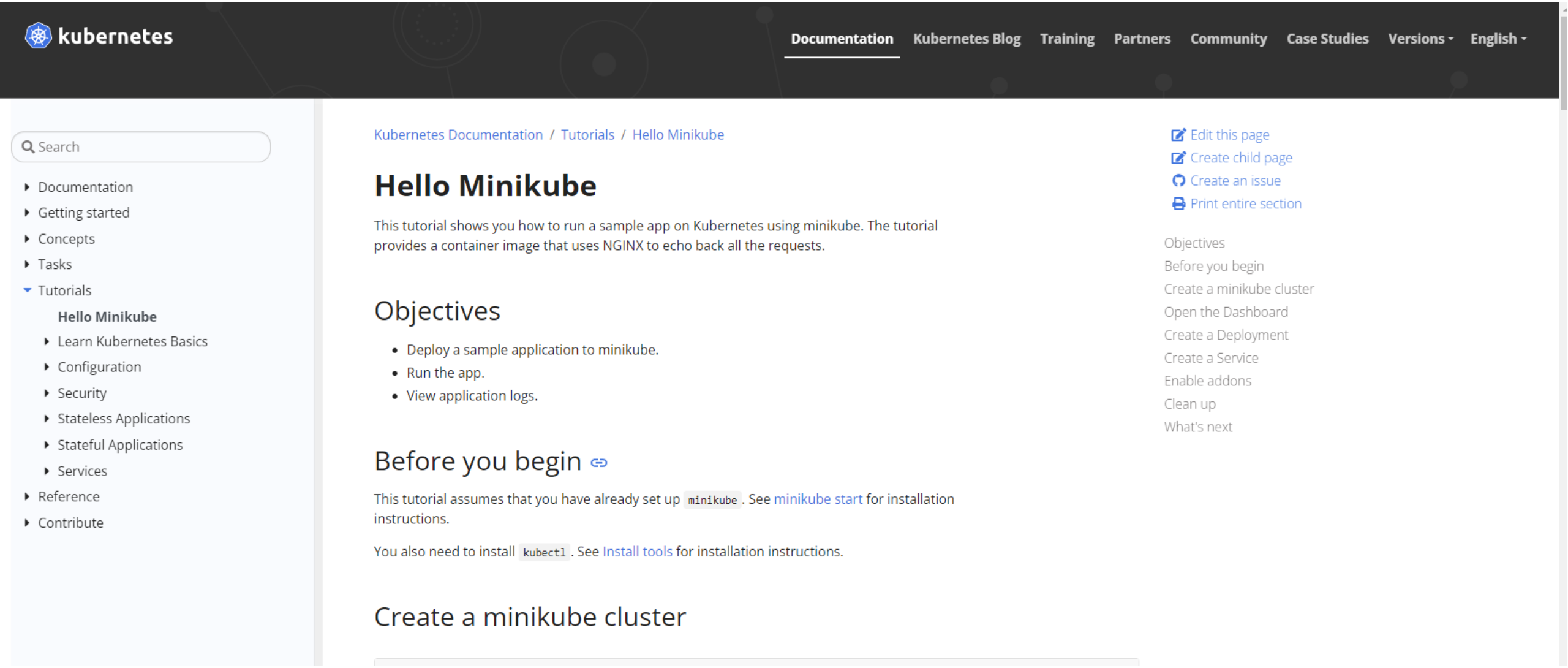
- `minikube start --kubernetes-version=stable --nodes=1 --cni=flannel`

- Minikube starten met Master Node en 2 Worker Nodes

- `minikube start --kubernetes-version=stable --nodes=2 --cni=flannel`

# MiniKube | Commando's

- **minikube start**: start een Minikube-cluster
- **minikube stop**: stopt de Minikube-cluster
- **minikube delete**: verwijdert de Minikube-cluster
- **minikube dashboard**: opent het Kubernetes-dashboard in de browser
- **minikube ssh**: opent een SSH-verbinding met de worker node
- **minikube status**: geeft de status van de Minikube-cluster weer
- **minikube addons list**: geeft een lijst weer van beschikbare add-ons voor Minikube
- **minikube addons enable** <addon-name>: schakelt de opgegeven add-on in
- **minikube addons disable** <addon-name>: schakelt de opgegeven add-on uit
- **minikube service** <service-name>: opent de service in de standaardbrowser
- **minikube logs**: geeft de logboeken weer van de Minikube-cluster




https://kubernetes.io/docs/tutorials/hello-minikube/

# Kubernetes Alternatieven
# K3S | K3d




SAXION
HOGESCHOOL

# K3S

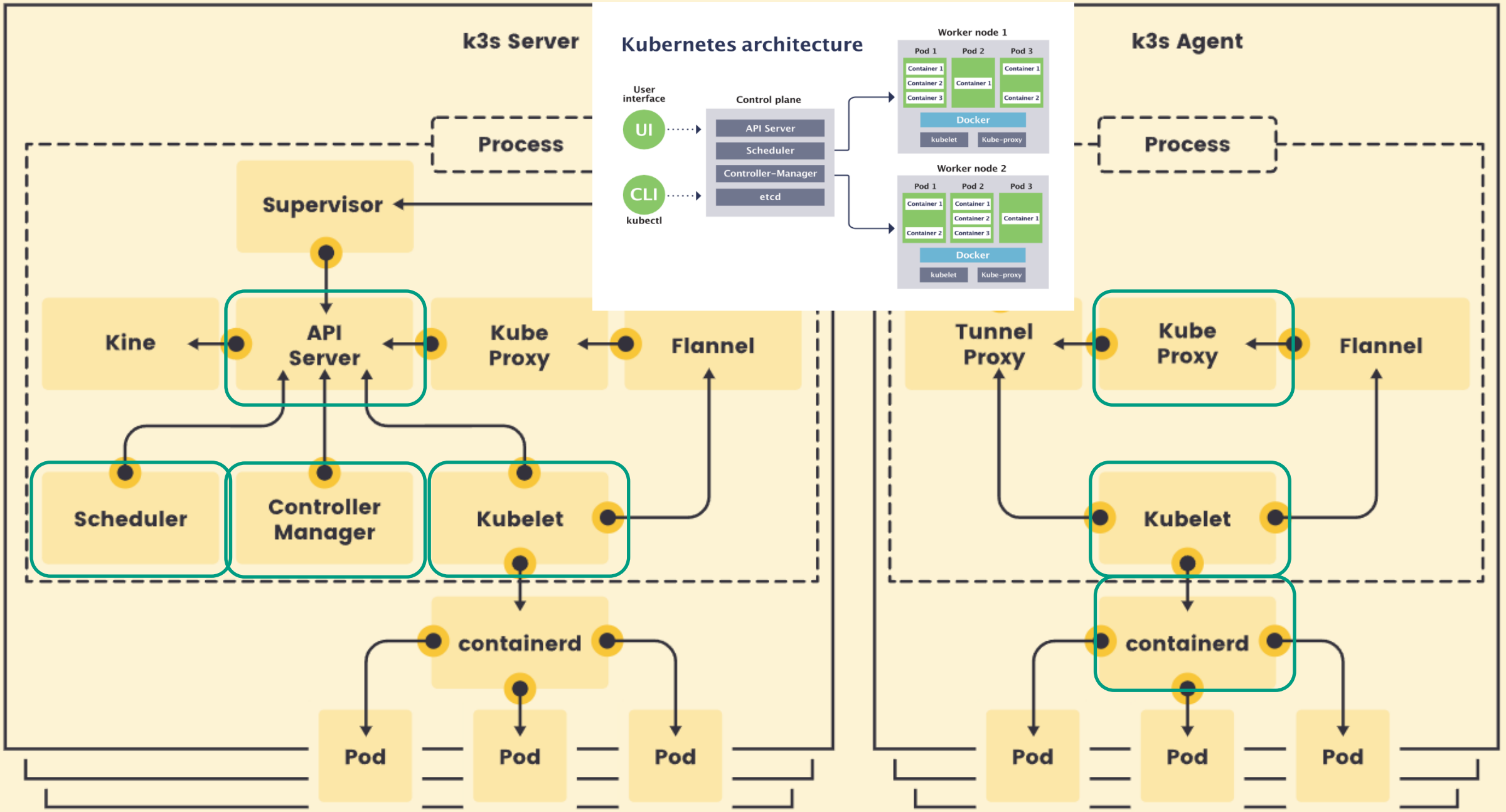
https://k3s.io/

- Is ontworpen als een **lichtgewicht, productiegereed Kubernetes-distributie** voor resource-beperkte omgevingen, zoals edge computing, IoT-apparaten, en kleine servers. Het is geoptimaliseerd voor minimale resourcevereisten en eenvoudige installatie en beheer, ook in productieomgevingen.
- Is extreem **lichtgewicht**. Het is ontworpen om te draaien op apparaten met beperkte middelen. De binaire bestandsgrootte is minder dan 100MB, en het vereist aanzienlijk minder RAM (vaak 512MB RAM en 1 CPU-kern als minimum). Dit maakt het ideaal voor Raspberry Pi's en vergelijkbare apparaten.

| Kenmerk | Minikube | K3s |
|---|---|---|
| **Doel** | Lokale ontwikkeling en testen | Lichtgewicht productie, edge, IoT, kleine servers |
| **Resourceverbruik** | Hoger (vaak VM-gebaseerd) | Erg laag (geoptimaliseerd voor beperkte hardware) |
| **Componenten** | Volledige Kubernetes-componenten (incl. etcd) | Vereenvoudigd (standaard SQLite, `containerd`, minder features) |
| **Installatie** | Eenvoudig, maar kan hypervisor-setup vereisen | Zeer eenvoudig en snel (enkele binary) |
| **Multi-node** | Primair één-node (beperkte multi-node support) | Volledige multi-node support (ook voor productie) |
| **Binaire grootte** | Groter | Zeer klein (onder 100MB) |

Bron:
Google Gemini

https://k3s.io/img/how-it-works-k3s-revised.svg



Overview

K3S
Maar dan
met
Docker
;-)

# Overview



## What is k3d?

k3d is a lightweight wrapper to run k3s (Rancher Lab's minimal Kubernetes distribution) in docker.

k3d makes it very easy to create single- and multi-node k3s clusters in docker, e.g. for local development on Kubernetes.

**Note:** k3d is a **community-driven project** but it's not an official Rancher (SUSE) product.
**Sponsoring**: To spend any significant amount of time improving k3d, we rely on sponsorships:

Table of contents

What is k3d?
Learning
Requirements
Releases
Installation
Install Script
Install current latest release
Install specific release
Other Installers
Quick Start
Connect
Related Projects

# Kubernetes Alternatieven MicroK8S




SAXION HOGESCHOOL

# MicroK8S

- MicroK8s is een volledig compatibele Kubernetes-distributie met een kleinere CPU- en geheugenfootprint dan de meeste andere.
- Het is ontworpen om een volledige Kubernetes-ervaring te bieden voor apparaten met beperkte rekenkracht en geheugen.
- MicroK8s heeft een aantal functies, waaronder:
  - **High availability:**
    - Een minimum van drie knooppunten is nodig om het controlepaneel voor hoge beschikbaarheid uit te voeren.
  - **Ingebouwde registry:**
    - MicroK8s bevat een ingebouwde Docker-registry die kan worden gebruikt om Docker-images op te slaan en te delen.
  - **Add-ons:**
    - MicroK8s wordt geleverd met verschillende add-ons, zoals DNS, dashboard, storage en meer.

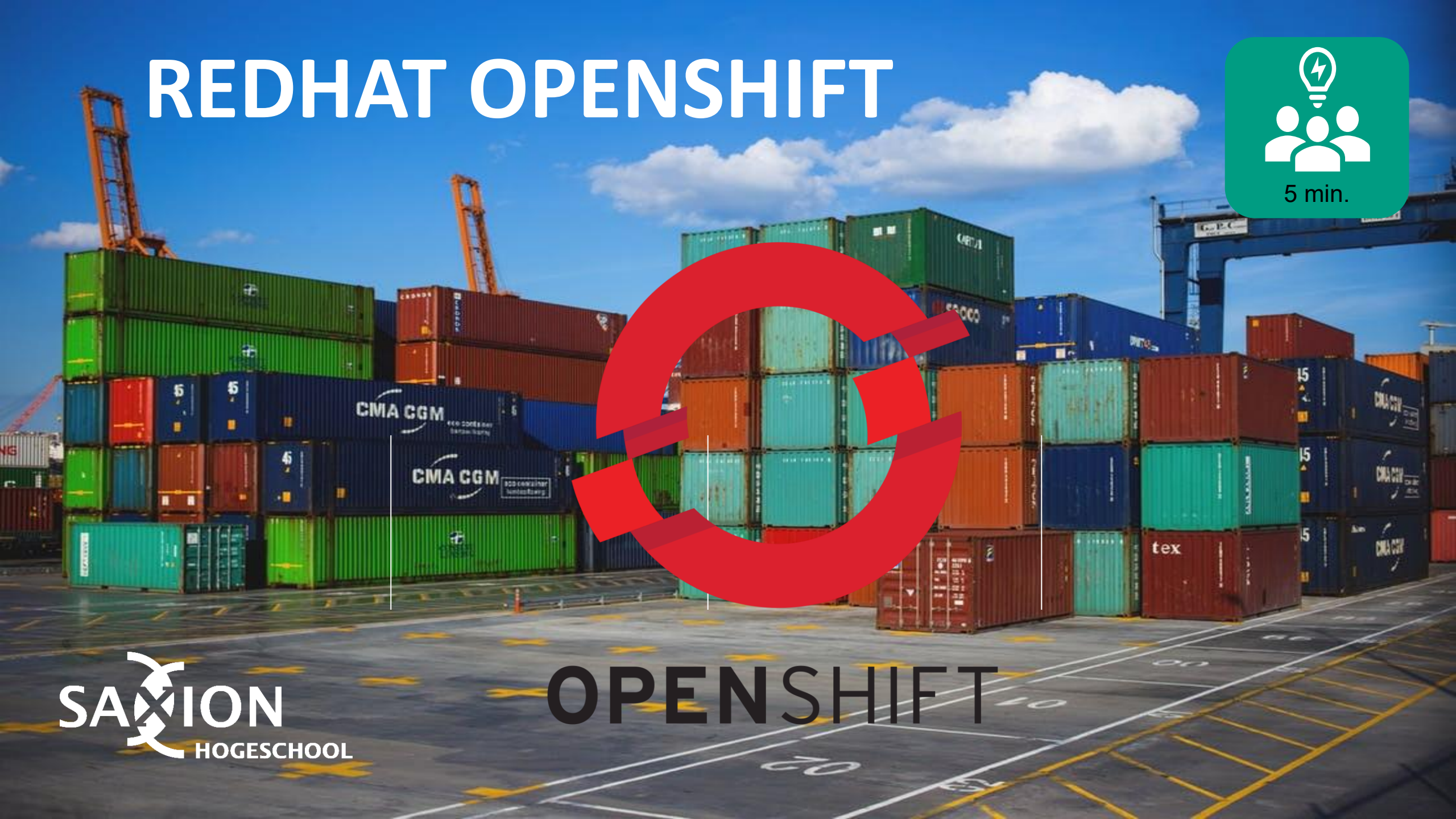
| Feature/Criterium | Minikube | K3s | MicroK8s |
|---|---|---|---|
| **Doel** | Lokale ontwikkeling en testen | Lichtgewicht productie, edge, IoT | Ontwikkeling, testen, kleine productie |
| **Installatie** | Vereist virtualisatie (VM, Docker Driver) | Eén binair bestand, eenvoudig te installeren | Eén snap-pakket, eenvoudig te installeren |
| **Voetafdruk** | Relatief zwaar (volle Kubernetes installatie) | Erg lichtgewicht, minimale benodigdheden | Lichtgewicht, maar iets zwaarder dan K3s |
| **Afhankelijkheden** | VirtualBox, Hyper-V, Docker Desktop, etc. | Weinig tot geen externe afhankelijkheden | Snapd |
| **Gebruiksgemak** | Redelijk eenvoudig voor lokale setup | Zeer eenvoudig voor snelle implementatie | Zeer eenvoudig door snap integratie |
| **Productierijp** | Minder geschikt voor productie | Zeer geschikt voor edge/IoT/kleine clusters | Goed voor kleine tot middelgrote productieomgevingen |
| **Add-ons** | Ingebouwde add-ons (dashboard, ingress) | Vereist handmatige toevoeging van CNI/storage | Ingebouwde add-ons (dashboard, DNS, storage) |

SAXION HOGESCHOOL

Bron: Google Gemini

# REDHAT OPENSHIFT

5 min.




OPENSHIFT

SAXION HOGESCHOOL

# RedHat OpenShift

- OpenShift gebruikt een gecertificeerde Kubernetes-distributie als de kern van zijn platform.
- OpenShift heeft de voorkeur voor CRI-O als container runtime interface, in plaats van Docker.
- OpenShift bevat een ingebouwde, veilige container-image registry, vaak gebaseerd op Red Hat Quay.
- OpenShift heeft zijn eigen Software Defined Networking (SDN) oplossing.

- Als Kubernetes de 'motor' is, dan is OpenShift de 'complete auto' met alle toeters en bellen, airconditioning, navigatie en een ingebouwd serviceplan, speciaal ontworpen voor de zakelijke markt.

Bron:
Google Gemini

# KubeVirt




10 min.

SAXION HOGESCHOOL

# KubeVirt

- **Wat is KubeVirt?**
  KubeVirt is een open-source extensie voor Kubernetes waarmee je niet alleen containers, maar ook virtuele machines (VM's) kunt beheren binnen hetzelfde platform.
- **Waarom gebruik je het?**
  Het combineert de voordelen van containers (zoals snelheid en schaalbaarheid) met de flexibiliteit van traditionele VM's, bijvoorbeeld voor legacy-applicaties die nog niet geschikt zijn voor containers.
- **Waar gebruik je het?**
  In hybride omgevingen waar zowel moderne containerapplicaties als oudere VM-gebaseerde systemen draaien, zoals in datacenters of bij migratieprojecten naar de cloud.
- **Voordeel voor DevOps/IT-beheer:**
  Je kunt alles beheren via één platform (Kubernetes), wat zorgt voor minder complexiteit, betere automatisering en efficiënter resourcegebruik.

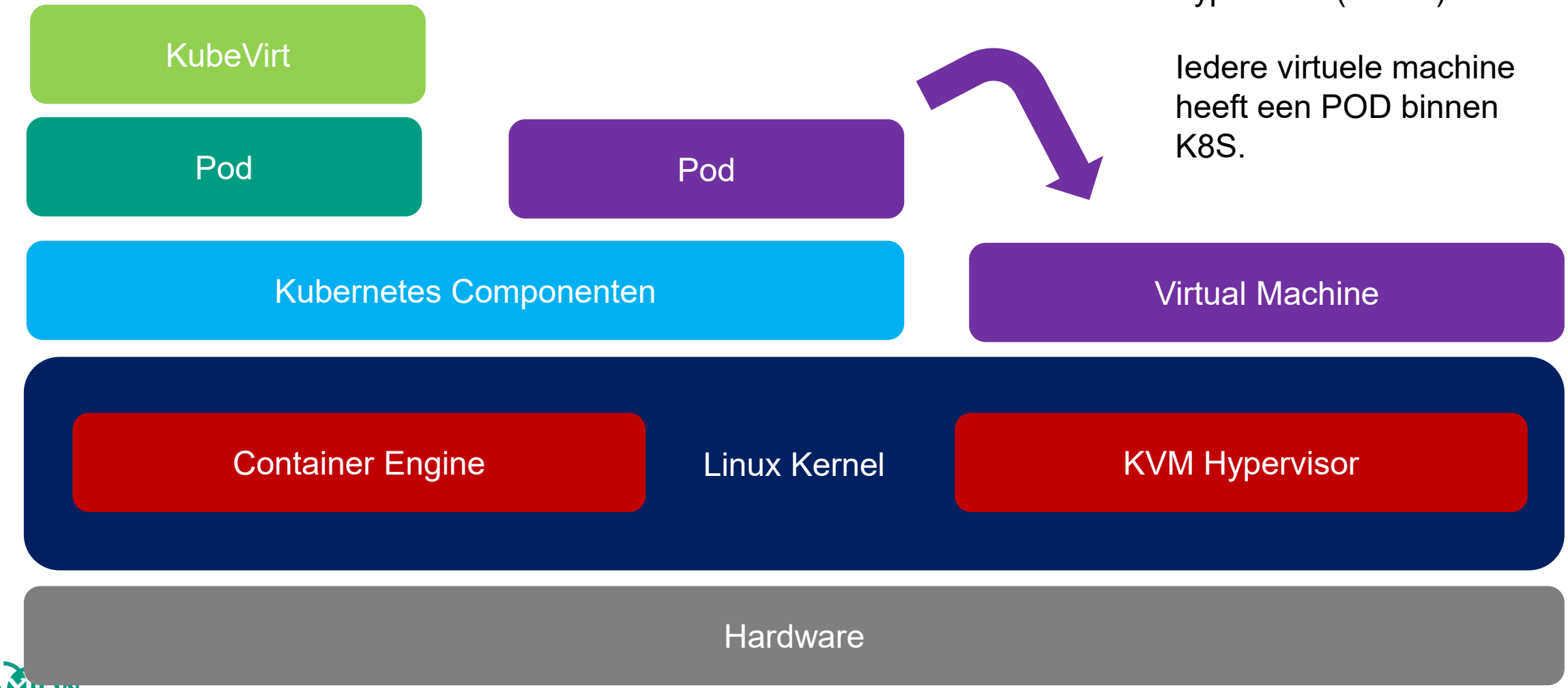


Bron:
CoPilot
https://www.redhat.com/en/topics/virtualization/what-is-kubevirt
https://www.redhat.com/en/blog/getting-started-with-kubevirt

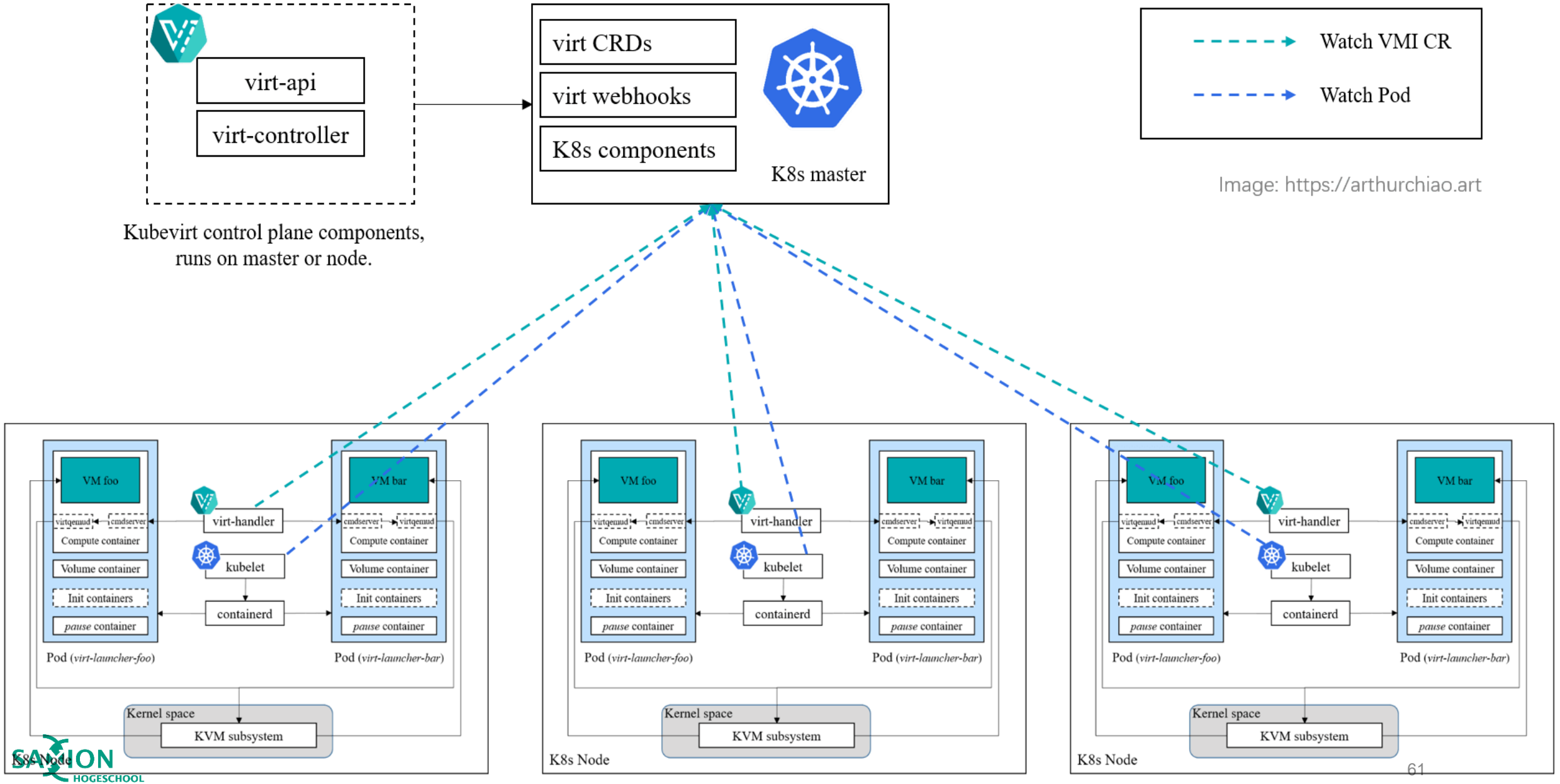
# Kubevirt | Achitectuur | Woker Node

Virtuele machine draait rechtstreeks op de hypervisor (kernel).

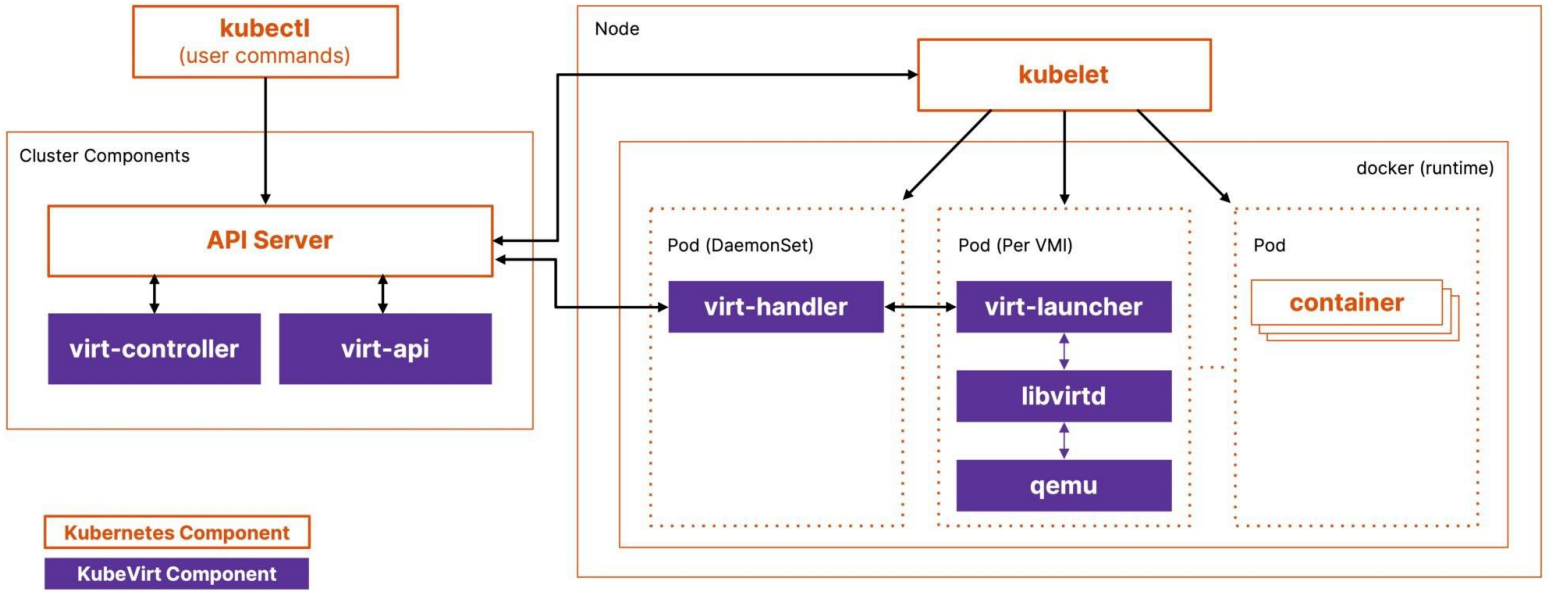
Iedere virtuele machine heeft een POD binnen K8S.




https://kubevirt.io/user-guide/architecture/




https://arthurchiao.art/assets/img/kubevirt-create-vm/kubevirt-arch.png




## Virt-Operator

**virt-operator** installs the required components and controllers to work with KubeVirt in Kubernetes. It also helps with rolling updates of KubeVirt's components and minimizes workload disruption.

## Virt-Controller

**virt-controller** manages the entire VM lifecycle, from initial power-on through operations like shutdown and reboot to eventual deletion. It orchestrates complex operations such as live migrations, coordinating between source and target nodes to ensure smooth transitions.

## Virt-Handler

**virt-handler** runs on each node as a DaemonSet, where a VM is scheduled and handles communication between the node and VM. It provides instructions to virt-launcher on launching the VM and the required dependencies. It also sends the telemetry data from VMs to virt-controller.

## Virt-Launcher

**virt-launcher** is a special pod for each VM that contains the actual process that runs the VM using QEMU. It ensures pod isolation and that each VM gets the required resources. Lastly, it updates the VM's status to the virt-handler.

https://portworx.com/knowledge-hub/understanding-kubevirt/

# KubeVirt

- Beschikbaar als addon op Minikube

# Aftekenen



Optie #2
AFTEKENEN

- Docker

Toepassen (10%)

- Je kan een Docker image maken op basis van een Dockerfile (dockerfile bestaat uit minimaal 5 van elkaar verschillende instructies);
- Je kan het zelfgemaakte image starten als container;
- Je kan verbinding maken met deze container via netwerk door gebruik van poorten;
- Je kan deze container stoppen en verwijderen;
- Je kan meerdere containers laten samenwerken door inzet van Docker Compose. Dit demonstreer met behulp van een webapplicatie die uit meerdere containers bestaat.



30 min

SAXION HOGESCHOOL

# Huiswerk

- Ga naar Brightspace
- Ga naar Les 6.1
- Ga naar Verwerking
- Doe opdrachten





2026 – 2027 | 2.1 Virtualisatie | Henk Broekman

Week 6 | Les 2

# Kubernetes



Container Orchestration
2 van 2

SAXION HOGESCHOOL

# Presentie



# Aftekenen



25 min

- Docker

Toepassen (10%)

- Je kan een Docker image maken op basis van een Dockerfile (dockerfile bestaat uit minimaal 5 van elkaar verschillende instructies);
- Je kan het zelfgemaakte image starten als container;
- Je kan verbinding maken met deze container via netwerk door gebruik van poorten;
- Je kan deze container stoppen en verwijderen;
- Je kan meerdere containers laten samenwerken door inzet van Docker Compose. Dit demonstreer met behulp van een webapplicatie die uit meerdere containers bestaat.



SAXION HOGESCHOOL

# Agenda



Kubernetes

Architectuur [recap]



Kubernetes

Objecten:
Replicaset
Deployment



Kubernetes

Service +
Opdracht

Storage



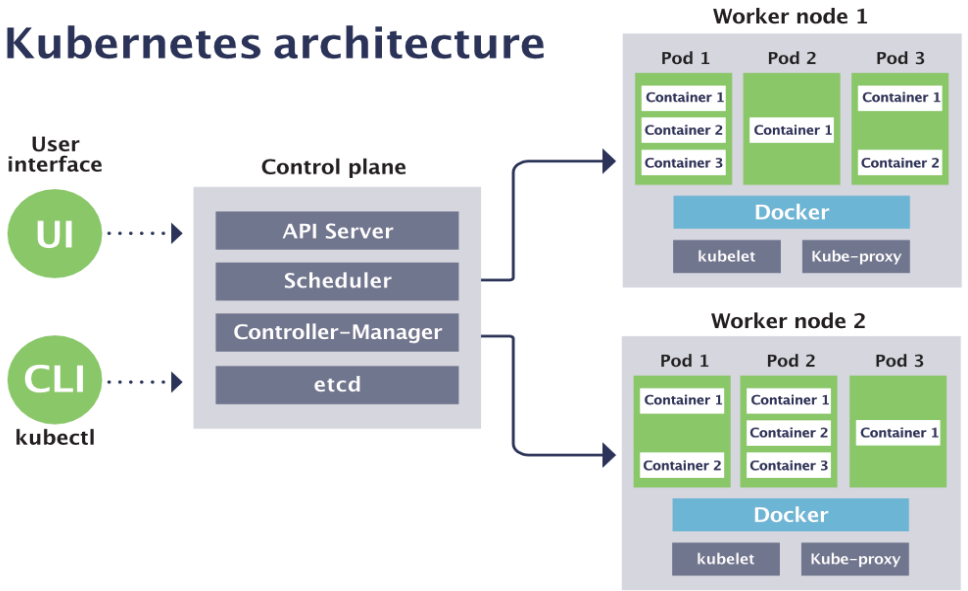
Kubernetes

Opdracht



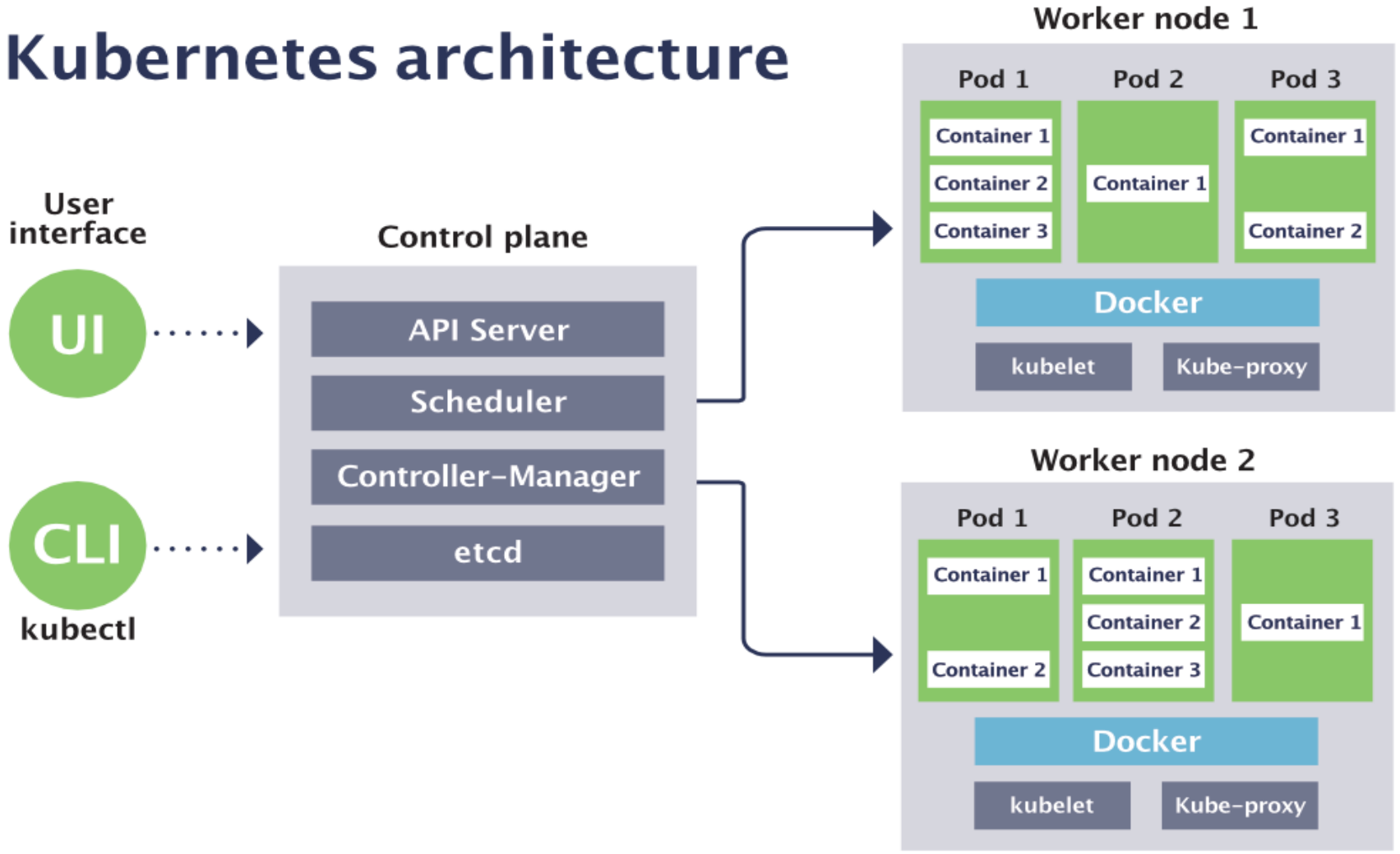

Techniek | Bachelor ICT | 2.1 Virtualisatie | Week 6 | L

# ARCHITECTUUR




SAXION HOGESCHOOL

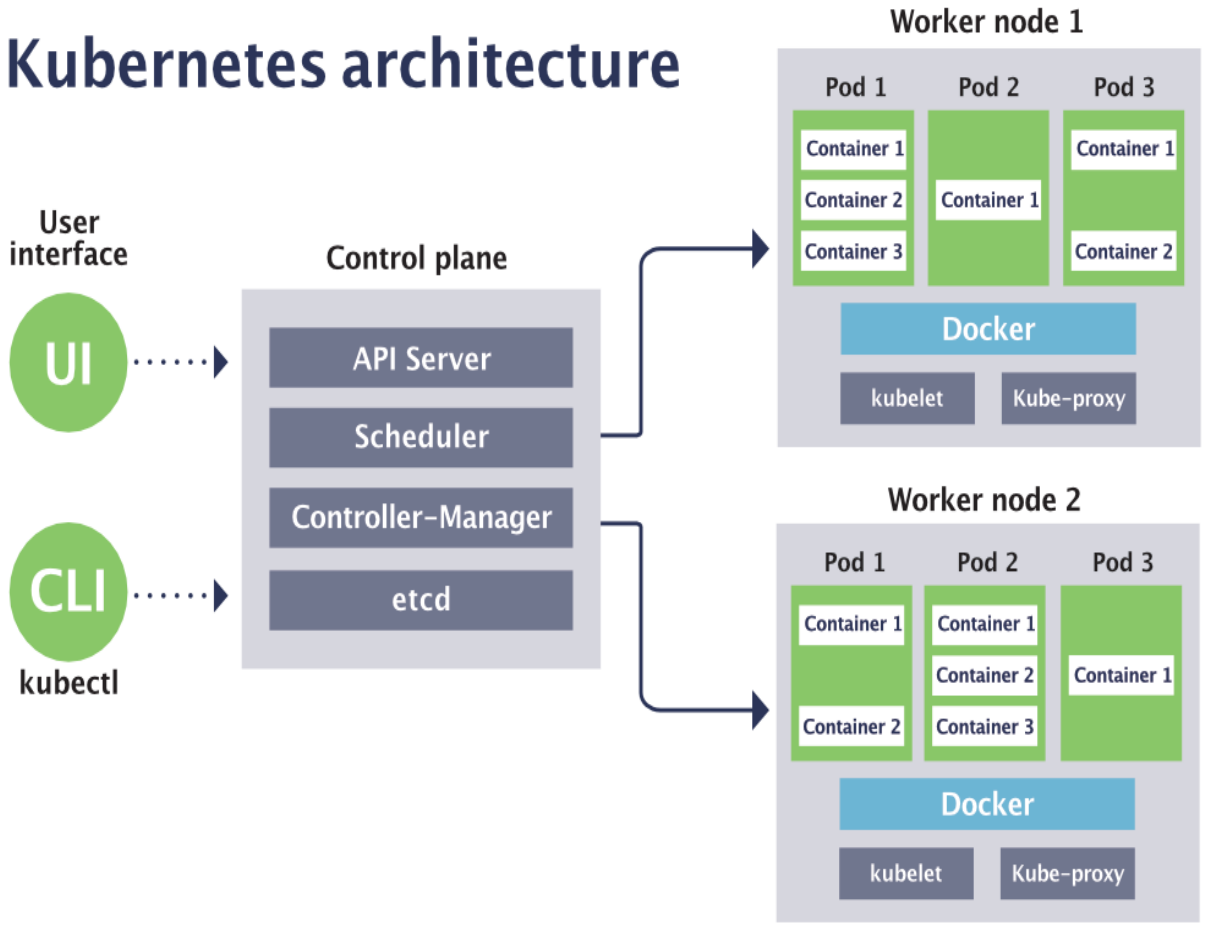
# Kubernetes | Architectuur | Details




https://images.ctfassets.net/w1bd7cq683kz/5Ex6830HzBPU5h8Ou8xQAB/2c948105fc10094348203bec6c1eab04/Kubernetes_20architecture_20diagram.png

# Kubernetes | Architectuur | Componenten | Master

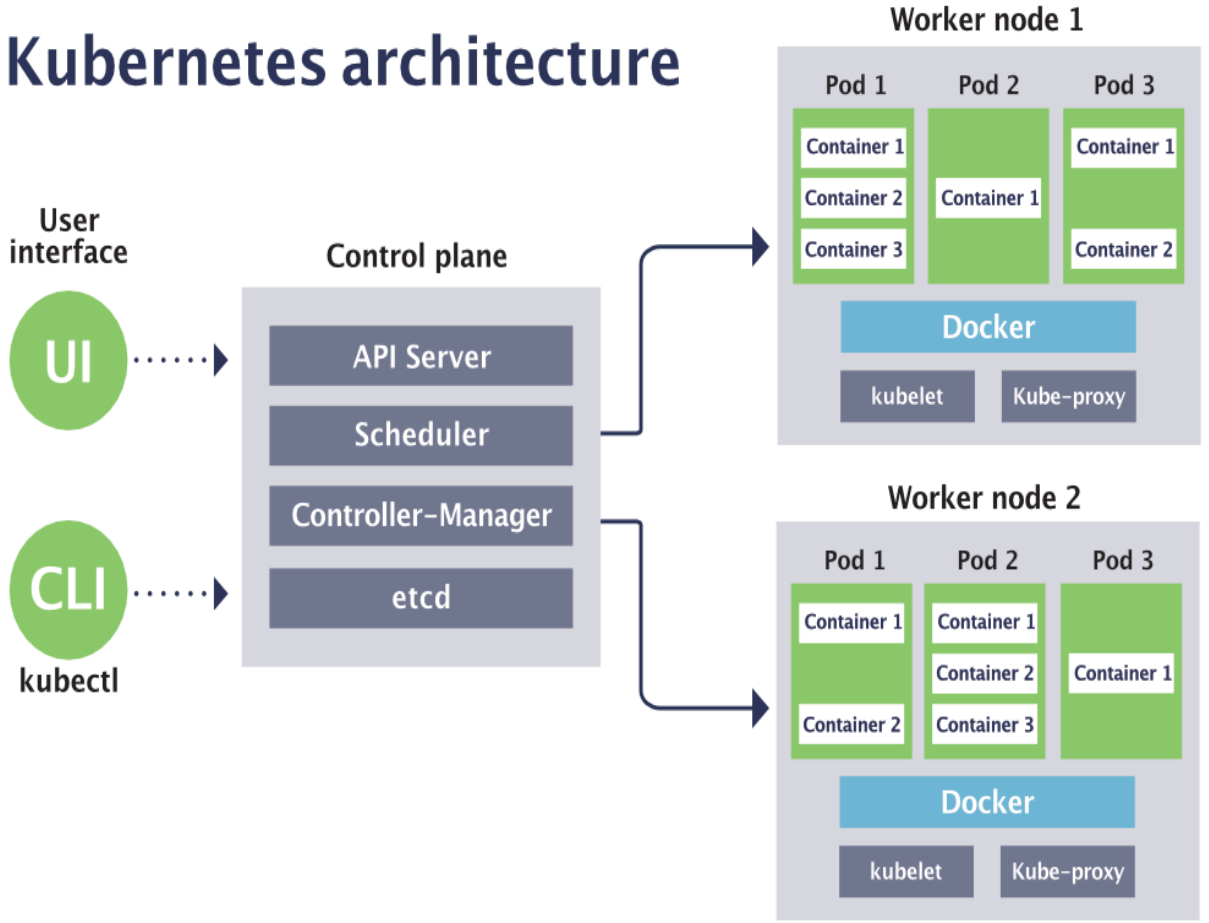
- **API Server**
  - The API Server is the front-end of the control plane and the only component in the control plane that we interact with directly. Internal system components, as well as external user components, all communicate via the same API.
- **Scheduler**
  - A Scheduler watches for new requests coming from the API Server and assigns them to healthy nodes.
  - It ranks the quality of the nodes and deploys pods to the best-suited node. If there are no suitable nodes, the pods are put in a pending state until such a node appears.
- **Controller**
  - The role of the Controller is to obtain the desired state from the API Server.
  - It checks the current state of the nodes.
  - It is tasked to control, and determines if there are any differences, and resolves them, if any.
- **etcd / Key-Value Store**
  - The Key-Value Store, also called etcd, is a database Kubernetes uses to back-up all cluster data.
  - It stores the entire configuration and state of the cluster.
  - The Master node queries etcd to retrieve parameters for the state of the nodes, pods, and containers.




https://images.ctfassets.net/w1bd7cq683kz/5Ex6830HzBPU5h8Ou8xQAB/2c948105fc10094348203bec6c1eab04/Kubernetes_20architecture_20diagram.png

# Kubernetes | Architectuur | Componenten | (Worker) Node

- **Kubelet**
  - The kubelet runs on every node in the cluster. It is the principal Kubernetes agent.
    - By installing kubelet, the node's CPU, RAM, and storage become part of the broader cluster. It watches for tasks sent from the API Server, executes the task, and reports back to the Master.
    - It also monitors pods and reports back to the control panel if a pod is not fully functional. Based on that information, the Master can then decide how to allocate tasks and resources to reach the desired state.
- **Container Runtime**
  - The container runtime pulls images from a container image registry and starts and stops containers. A 3rd party software or plugin, such as Docker, usually performs this function.
- **Kube-proxy**
  - The kube-proxy makes sure that each node gets its IP address, implements local iptables and rules to handle routing and traffic load-balancing.




https://images.ctfassets.net/w1bd7cq683kz/5Ex6830HzBPU5h8Ou8xQAB/2c948105fc10094348203bec6c1eab04/Kubernetes_20architecture_20diagram.png

Search

- Documentation
- Getting started
- Concepts
- Tasks
- Tutorials
  - Hello Minikube
  - **Learn Kubernetes Basics**
    - Create a Cluster
    - Deploy an App
    - Explore Your App
    - Expose Your App Publicly
    - Scale Your App
    - Update Your App
  - Configuration
  - Security
  - Stateless Applications
  - Stateful Applications
  - Services
- Reference
- Contribute



Edit this page
Create child page
Create an issue
Print entire section

# Learn Kubernetes Basics

## Kubernetes Basics

This tutorial provides a walkthrough of the basics of the Kubernetes cluster orchestration system. Each module contains some background information on major Kubernetes features and concepts, and a tutorial for you to follow along.

Using the tutorials, you can learn to:

- Deploy a containerized application on a cluster.
- Scale the deployment.
- Update the containerized application with a new software version.
- Debug the containerized application.

## What can Kubernetes do for you?

With modern web services, users expect applications to be available 24/7, and developers expect to deploy new versions of those applications several times a day. Containerization helps package software to serve these goals, enabling applications to be released and updated without downtime. Kubernetes helps you make sure those containerized applications run where and when you want, and helps them find the resources and tools they need to work. Kubernetes is a production-ready, open source platform designed with Google's accumulated experience in container orchestration, combined with best-of-breed ideas from the community.



https://kubernetes.io/docs/tutorials/kubernetes-basics/

# Kubernetes Objecten

- **Workload: Pod**
  - Een pod is de *kleinste en eenvoudigste eenheid* in de Kubernetes-objectenhiërarchie.
  - Het is een enkele instantie van een applicatie in Kubernetes, die één of meer containers kan bevatten.
  - Een pod kan worden gebruikt om één of meer containers te starten en te beheren.
- **Workload: ReplicaSet**
  - Een ReplicaSet is een Kubernetes-object dat ervoor zorgt dat een bepaald aantal pod-replica's op elk moment wordt uitgevoerd.
  - Het is echter een hoger niveau concept dan een Pod en biedt declaratieve updates aan Pods samen met veel andere nuttige functies.
- **Workload: Deployment**
  - Een Deployment is een *hoger niveau concept dan ReplicaSets* en biedt declaratieve updates aan Pods samen met veel andere nuttige functies.
  - Het beheert ReplicaSets en zorgt voor declaratieve updates aan Pods.
- **Service:**
  - Een Service is een abstractie die definieert hoe Pods toegankelijk zijn binnen het cluster.
  - Het biedt een stabiel IP-adres en DNS-naam voor de set Pods die het vertegenwoordigt.

https://kubernetes.io/docs/concepts/workloads/
https://kubernetes.io/docs/concepts/services-networking/service/

# Workload: Replicaset



5 min.

kubernetes

SAXION HOGESCHOOL

https://kubernetes.io/docs/concepts/workloads/controllers/replicaset/

# Workload: Replicaset

- **Replicatie en hoge beschikbaarheid:**
  - Een replicaset biedt functionaliteit voor het beheren van replica's van pods in een Kubernetes-cluster.
  - Het zorgt ervoor dat het gewenste aantal replica's van een pod altijd actief en beschikbaar is, waardoor hoge beschikbaarheid van applicaties wordt gegarandeerd.
- **Automatische schaalbaarheid:**
  - Een replicaset kan worden geconfigureerd om automatisch het aantal replica's op- of af te schalen op basis van belastingscriteria of configuratie-instellingen.
  - Hierdoor kan de capaciteit van een applicatie dynamisch worden aangepast aan de vraag, wat resulteert in efficiënt gebruik van resources en optimale prestaties.

# Workload: Replicaset

- **Herstelmechanismen:**
  - Een replicaset bewaakt de status van de pods en kan automatisch herstelmechanismen activeren in geval van een storing.
  - Als een pod faalt, wordt deze automatisch opnieuw gestart of vervangen door een nieuwe replica om ervoor te zorgen dat de applicatie blijft draaien zonder onderbrekingen.
- **Versiebeheer en updates:**
  - Een replicaset ondersteunt het beheer van verschillende versies van een applicatie.
  - Hierdoor kunnen updates en rollbacks van applicatieversies worden uitgevoerd, waarbij nieuwe replica's geleidelijk worden uitgerold en oude replica's worden verwijderd.
  - Dit maakt het mogelijk om wijzigingen in applicaties gecontroleerd en zonder impact op de gebruikerservaring door te voeren.

# Workload: Replicaset

- Dit is een voorbeeld van een configuratiebestand voor het implementeren van een **nginx**-container met **3 replica's**.
- Het bestand bevat de volgende secties:
  - **apiVersion**: de versie van de Kubernetes API die wordt gebruikt.
  - **kind**: het type object dat wordt gemaakt.
  - **metadata**: informatie over het object, zoals de naam.
  - **spec**: de specificatie van het object, zoals het aantal replica's en de sjabloon voor de pods.
  - **selector**: de labels die worden gebruikt om de pods te selecteren die worden beheerd door deze replicaset.
  - **template**: de sjabloon voor de pods die worden gemaakt.

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
        - containerPort: 80
```

## https://raw.githubusercontent.com/kubernetes/website/main/content/en/examples/controllers/frontend.yaml

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: frontend
  labels:
    app: guestbook
    tier: frontend
spec:
  # modify replicas according to your case
  replicas: 3
  selector:
    matchLabels:
      tier: frontend
  template:
    metadata:
      labels:
        tier: frontend
    spec:
      containers:
      - name: php-redis
        image: gcr.io/google_samples/gb-frontend:v3
```

```
NAME       DESIRED   CURRENT   READY   AGE
frontend   3         3         3       6s
```

# Workload: Deployment



6 min.

kubernetes

SAXION HOGESCHOOL

https://kubernetes.io/docs/concepts/workloads/controllers/deployment/

# Workload: Deployment

- **Beheer van replicatie:**
  - Een deployment biedt functionaliteit voor het beheren van replicatie van applicaties.
  - Het definieert het *gewenste aantal replica's* van een applicatie en zorgt ervoor dat het aantal actieve replica's overeenkomt met de gewenste staat (desired state).
- **Rollende updates:**
  - Met behulp van een deployment kunnen rollende updates worden uitgevoerd.
  - *Dit betekent dat nieuwe versies van een applicatie geleidelijk worden uitgerold, waarbij oude replica's geleidelijk worden vervangen door nieuwe replica's*.
  - Dit zorgt voor een soepele overgang zonder onderbreking van de service.
- **Rollbacks:**
  - In geval van problemen met een implementatie biedt een deployment de mogelijkheid om rollbacks uit te voeren.
  - Als er bijvoorbeeld een fout optreedt na een update, kan de deployment snel en eenvoudig worden teruggedraaid naar een vorige stabiele versie van de applicatie.

# Workload: Deployment

- **Health checks en automatische herstelmechanismen:**
  - Een deployment kan health checks instellen om de status van pods en containers te controleren.
  - Als een pod of container wordt gedetecteerd als ongezond, kan de deployment automatisch herstelmechanismen activeren, zoals het opnieuw opstarten van de pod of het vervangen van de ongezonde pod door een nieuwe replica.
- **Schaalbaarheid:**
  - Een deployment biedt mogelijkheden voor schaalbaarheid van applicaties.
  - Het kan worden geconfigureerd om automatisch het aantal replica's op- of af te schalen op basis van de belasting of configuratie-instellingen.
  - Dit zorgt voor flexibiliteit en optimalisatie van de resource-allocatie binnen het Kubernetes-cluster.

# Workload: Deployment

- Dit is een voorbeeld van een configuratiebestand voor het implementeren van een **nginx**-container met **3 replica's**.
- Het bestand bevat de volgende secties:
  - **apiVersion**: de versie van de Kubernetes API die wordt gebruikt.
  - **kind**: het type object dat wordt gemaakt.
  - **metadata**: informatie over het object, zoals de naam.
  - **spec**: de specificatie van het object, zoals het aantal replica's en de sjabloon voor de pods.
  - **selector**: de labels die worden gebruikt om de pods te selecteren die worden beheerd door deze implementatie.
  - **template**: de sjabloon voor de pods die worden gemaakt.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
        - containerPort: 80
```

## https://raw.githubusercontent.com/kubernetes/website/main/content/en/examples/controllers/nginx-deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
        - containerPort: 80
```

```
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment   3/3     3            3           18s
```

# Handige website




Kubernetes YAML Generator - Powered by Octopus. Learn more...

EDIT YAML | Dark Theme

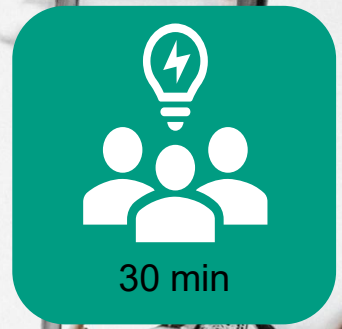
| | |
|---|---|
| **Resource Type** | Step configures a **Deployment** |
| **Deployment** | Create a Deployment with **1** replicas and the label **app: web** |
| **Deployment Strategy** | Update the Deployment with the **rolling deployment** strategy |
| **Volumes** | No volumes have been included (default) |
| **Containers** | Deploy image **nginx** exposing port: **80:TCP** |
| **DNS Policy** | No DNS policy specified (default) |
| **DNS Config** | No DNS configuration options specified. (default) |
| **Host Networking** | Host network access set to **True** |
| **Pod Security Context** | No pod security settings configured. (default) |
| **Pod Affinity / Anti-Affinity** | **Preferred** anti-affinity with weight **100** matching topology key **kubernetes.io/hostname** label key **app** is **in** value **web** |
| **Node Affinity** | No affinity rules defined (default) |
| **Tolerations** | No tolerations defined (default) |
| **Pod Annotations** | No annotations have been included (default) |
| **Deployment Annotations** | No annotations have been included (default) |
| **Service Account Name** | No pod service account specified (default) |

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: octopus-deployment
  labels:
    app: web
spec:
  selector:
    matchLabels:
      octopusexport: OctopusExport
  replicas: 1
  strategy:
    type: RollingUpdate
  template:
    metadata:
      labels:
        app: web
        octopusexport: OctopusExport
    spec:
      hostNetwork: true
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80
      affinity:
        podAntiAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
            - weight: 100
              podAffinityTerm:
                labelSelector:
                  matchExpressions:
                    - key: app
                      operator: In
                      values:
                        - web
                topologyKey: kubernetes.io/hostname
```

# Opdracht



- Start eigen Kubernetes omgeving.
  - Docker Desktop
  - Docker Desktop met Minikube
  - Play with Kubernetes website (inlog GIT account)
- Start Kubernetes Cluster met 2 Worker Nodes
- Geef commando:
  - kubectl apply -f https://raw.githubusercontent.com/kubernetes/website/main/content/en/examples/application/deployment.yaml
  - kubectl describe deployment nginx-deployment
  - kubectl get pods
  - kubectl describe pod [naam van de POD]




```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
        - containerPort: 80
```

SAXION HOGESCHOOL

https://kubernetes.io/docs/tasks/run-application/run-stateless-application-deployment/

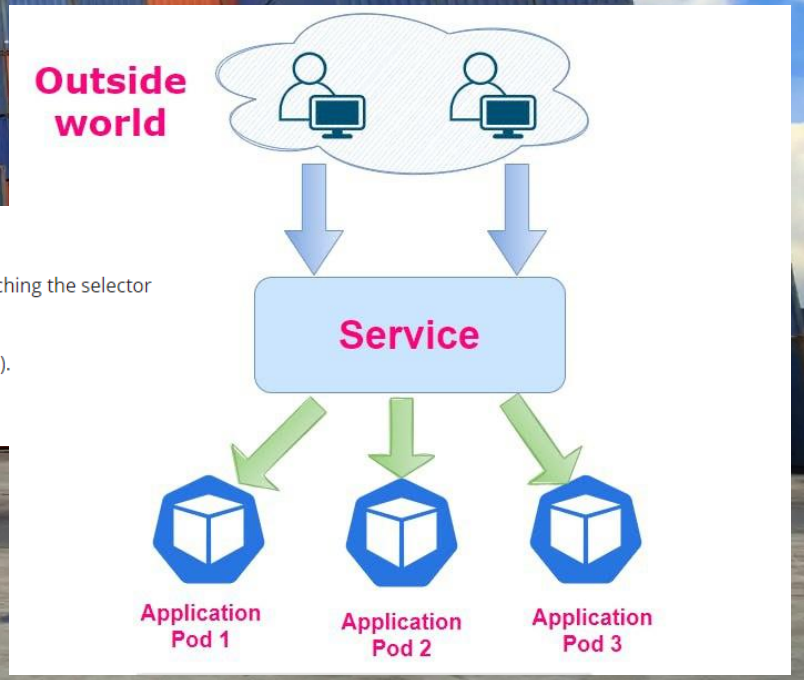
# service

35 min.

- **Service**
  A method for exposing a network application that is running as one or more Pods in your cluster.[-]
  The set of Pods targeted by a Service is (usually) determined by a selector. If more Pods are added or removed, the set of Pods matching the selector will change. The Service makes sure that network traffic can be directed to the current set of Pods for the workload.

  Kubernetes Services either use IP networking (IPv4, IPv6, or both), or reference an external name in the Domain Name System (DNS).

  The Service abstraction enables other mechanisms, such as Ingress and Gateway.




https://cdn.hashnode.com/res/hashnode/image/upload/v1627896838293/XhLiNARNO.png?auto=compress,format&format=webp

SAXION HOGESCHOOL

# Kubernetes | Service |

## Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-nginx
spec:
  selector:
    matchLabels:
      run: my-nginx
  replicas: 3
  template:
    metadata:
      labels:
        run: my-nginx
    spec:
      containers:
        - name: my-nginx
          image: nginx
          ports:
            - containerPort: 80

$ kubectl apply -f nginx.yml
deployment.apps/my-nginx created
```

## Resultaat

```
$ kubectl get deploy
NAME       READY   UP-TO-DATE   AVAILABLE   AGE
my-nginx   3/3     3            3           4m6s

$ kubectl get pods -o wide
NAME                        READY   STATUS    RESTARTS   AGE     IP
NODE                                              NOMINATED NODE
READINESS GATES

my-nginx-5b56ccd65f-dkqw9   1/1     Running   0          90s
10.0.2.116   ip-10-0-2-96.ap-southeast-2.compute.internal    <none>
<none>
my-nginx-5b56ccd65f-sndqv   1/1     Running   0          90s
10.0.1.97    ip-10-0-1-193.ap-southeast-2.compute.internal   <none>
<none>
my-nginx-5b56ccd65f-w26hx   1/1     Running   0          90s
10.0.2.246   ip-10-0-2-66.ap-southeast-2.compute.internal    <none>
<none>
```

PROBLEEM !!!

IP-adressen zijn steeds anders !

Oplossing: SERVICE

```
$ kubectl exec -it my-nginx-5b56ccd65f-dkqw9 -- bash

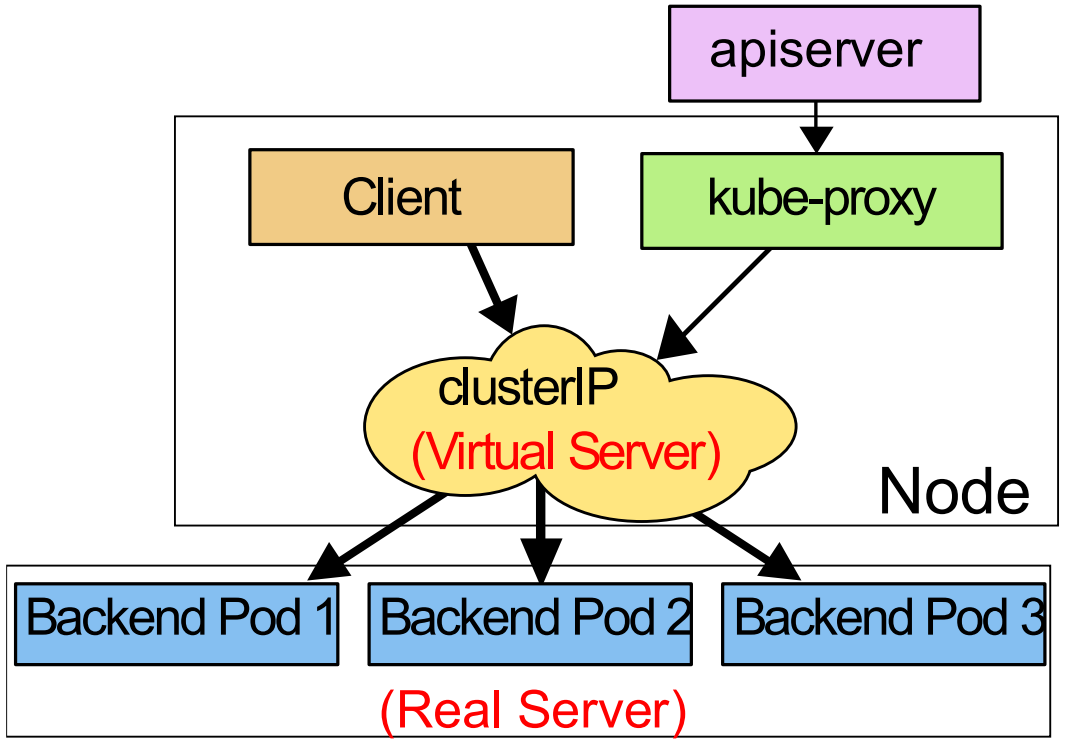
# ping 10.0.1.97
PING 10.0.1.97 (10.0.1.97) 56(84) bytes of data.
64 bytes from 10.0.1.97: icmp_seq=1 ttl=253 time=0.823 ms
64 bytes from 10.0.1.97: icmp_seq=2 ttl=253 time=0.688 ms
64 bytes from 10.0.1.97: icmp_seq=3 ttl=253 time=0.754 ms

# ping 10.0.2.246
PING 10.0.2.246 (10.0.2.246) 56(84) bytes of data.
64 bytes from 10.0.2.246: icmp_seq=1 ttl=253 time=0.364 ms
64 bytes from 10.0.2.246: icmp_seq=2 ttl=253 time=0.203 ms
64 bytes from 10.0.2.246: icmp_seq=3 ttl=253 time=0.207 ms
```

https://medium.com/@zhaoyi0113/kubernetes-how-does-service-network-work-in-the-cluster-d235b69ff536

# Services

- A **Service** provides an abstract way to expose an application running on a set of Pods as network service to the rest of the world
- Since Pods are ephemeral, services allow users to access the backend applications via a common way
- Service types are:
  - ClusterIP: Exposes the service on a cluster-internal IP
  - NodePort: Exposes the service on each Node's IP at a static port
  - LoadBalancer: Exposes the service externally and loadbalances it
  - ExternalName: Maps the service to a name, returns a CNAME record

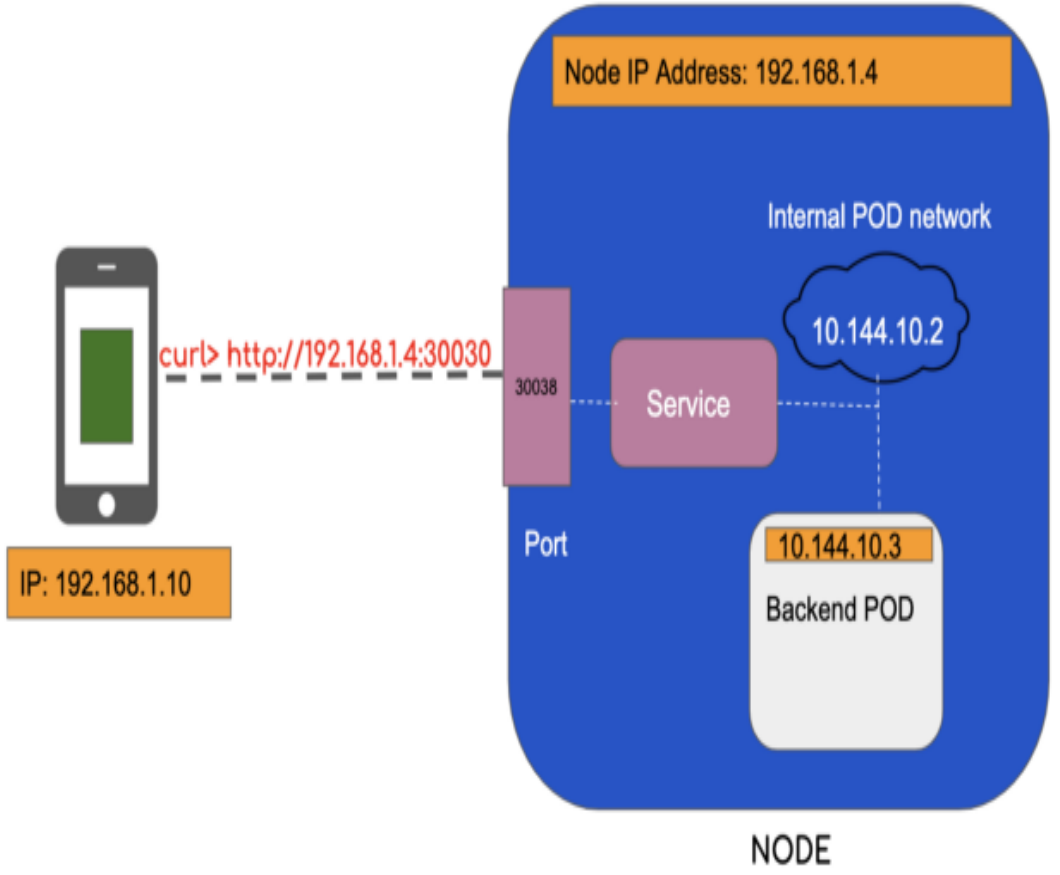



Reference:
https://kubernetes.io/docs/concepts/services-networking/service/

# Kubernetes | Service |

https://kubernetes.io/docs/concepts/services-networking/service/

- Service, is another type of object in Kubernetes, provides network stable abstraction in front of your pods.
- Each service has a name and a IP ,
- Kubernetes cluster guarantees that the name and ip in the service entire lifecycle never change.
- That how it makes service as a stable network abstraction.
- Meer info:
  - https://kubernetes.io/docs/concepts/services-networking/service/




https://medium.com/@zhaoyi0113/kubernetes-how-does-service-network-work-in-the-cluster-d235b69ff536
https://miro.medium.com/max/720/1*NGwqGzdO4BJCMItmKakCrQ.png

SAXION HOGESCHOOL

# Kubernetes | Service | YAML |

```
apiVersion: v1
kind: Service
metadata:
  name: my-nginx
  labels:
    run: my-nginx
spec:
  ports:
  - port: 80
    protocol: TCP
  selector:
    run: my-nginx


$ kubectl get service
NAME                TYPE        CLUSTER-IP       EXTERNAL-IP
PORT(S)                       AGE
my-nginx            ClusterIP   172.20.16.40     <none>        80/TCP
5s
```

https://medium.com/@zhaoyi0113/kubernetes-how-does-service-network-work-in-the-cluster-d235b69ff536

- Selector geeft aan welke pods gekoppeld worden aan service
  - Hier dus pods met label run : my-nginx.
- Deze worden hierna gecreerd door de deployment.

# Kubernetes | Service | Exposing Services |

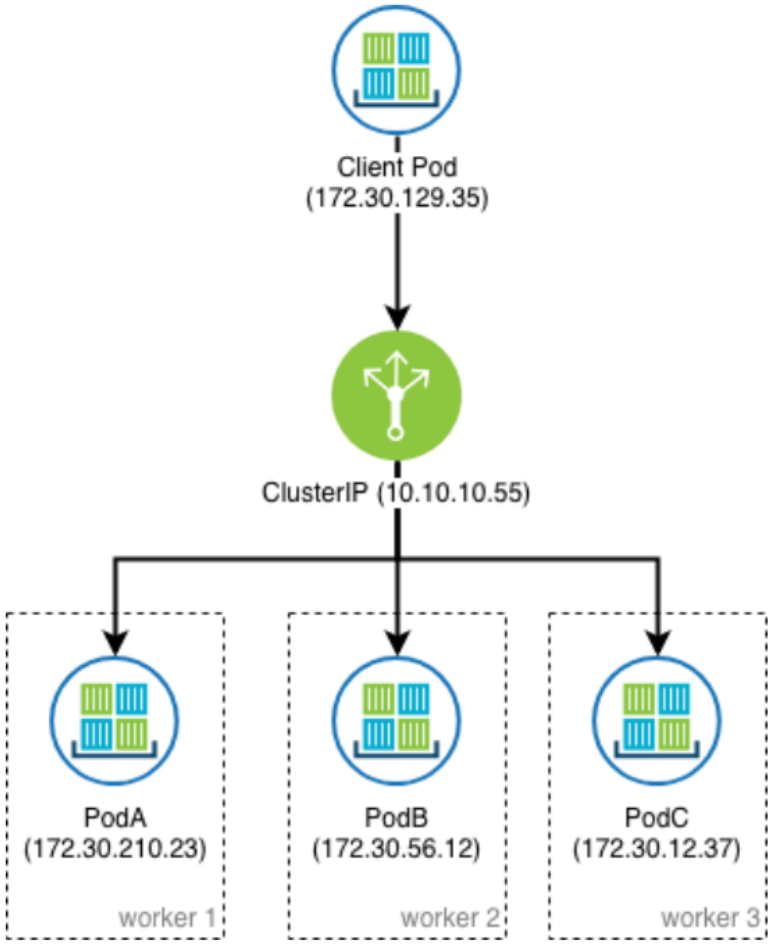
```
apiVersion: v1
kind: Service
metadata:
  name: my-nginx
  labels:
    run: my-nginx
spec:
  type: NodePort
  ports:
  - port: 80
    protocol: TCP
    nodePort: 30000
  selector:
    run: my-nginx
```

- Type bepaalt hoe service beschikbaar wordt gesteld

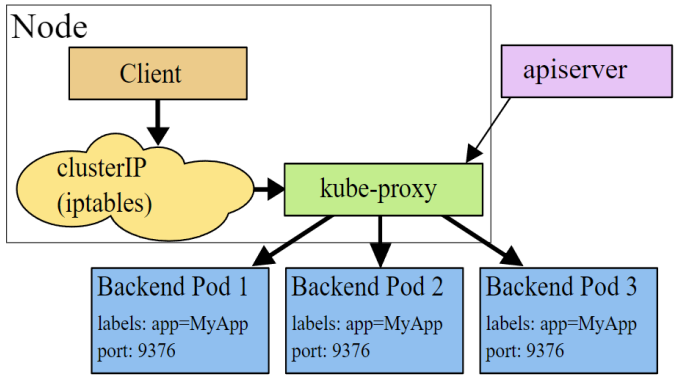
- 4 opties:
  - ClusterIP (default)
  - NodePort
  - LoadBalancer
  - ExternalName

https://medium.com/@zhaoyi0113/kubernetes-how-does-service-network-work-in-the-cluster-d235b69ff536

# Kubernetes | Service | Exposing Services | ClusterIP



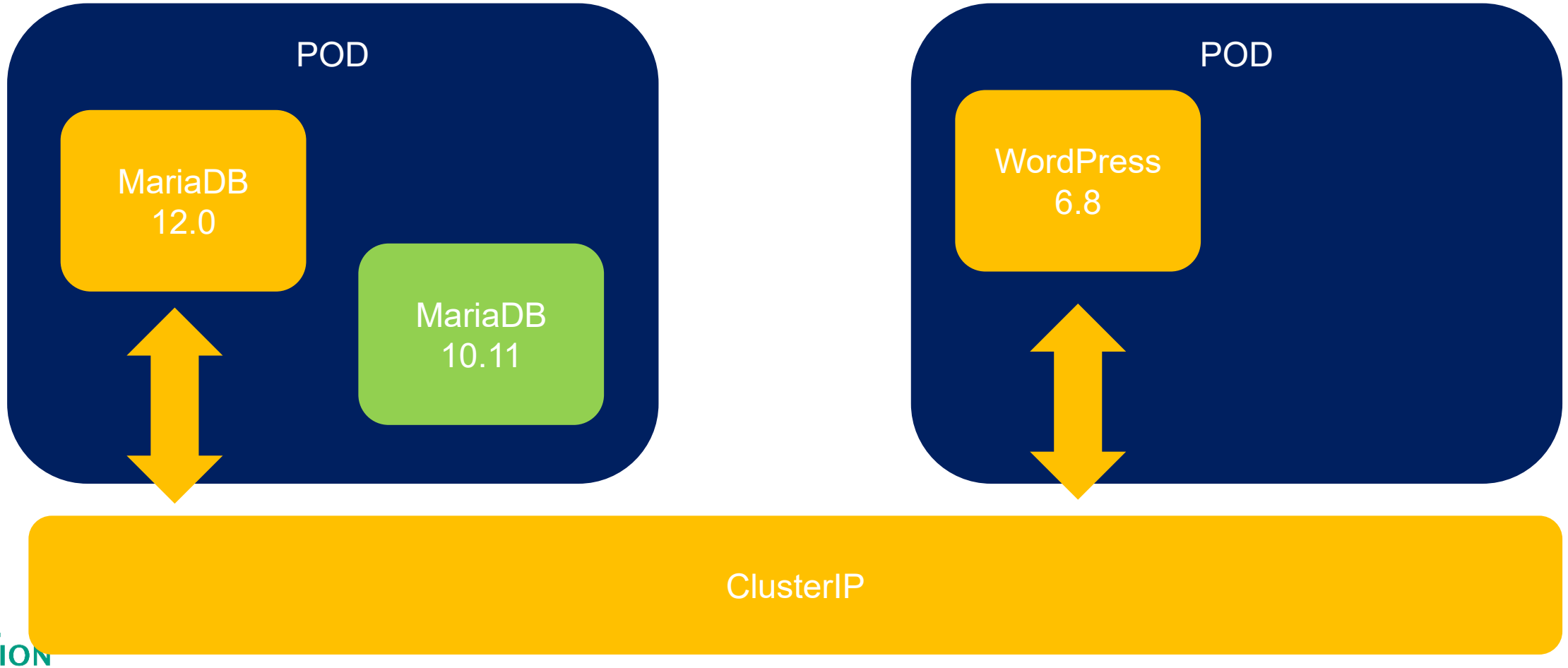

- ClusterIP (default)
  - Exposes the Service on an internal IP in the cluster.
  - This type makes the Service only reachable from within the cluster.
- Bron:
  - https://kubernetes.io/docs/tutorials/kubernetes-basics/expose/expose-intro/



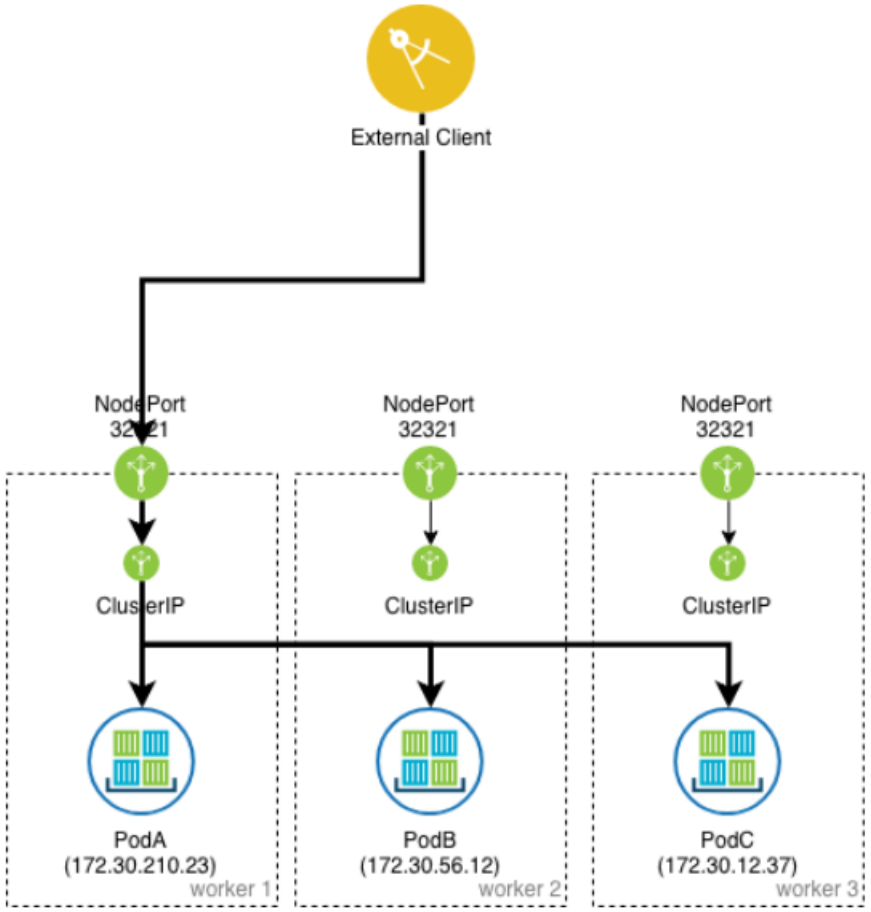

https://kubernetes.io/docs/concepts/services-networking/service/#type-clusterip

# Voorbeeld

MariaDB 12.0 en Wordpress 6.8
Samen 1 CMS omgeving



# Kubernetes | Deployment | Service | Exposing Services | NodePort

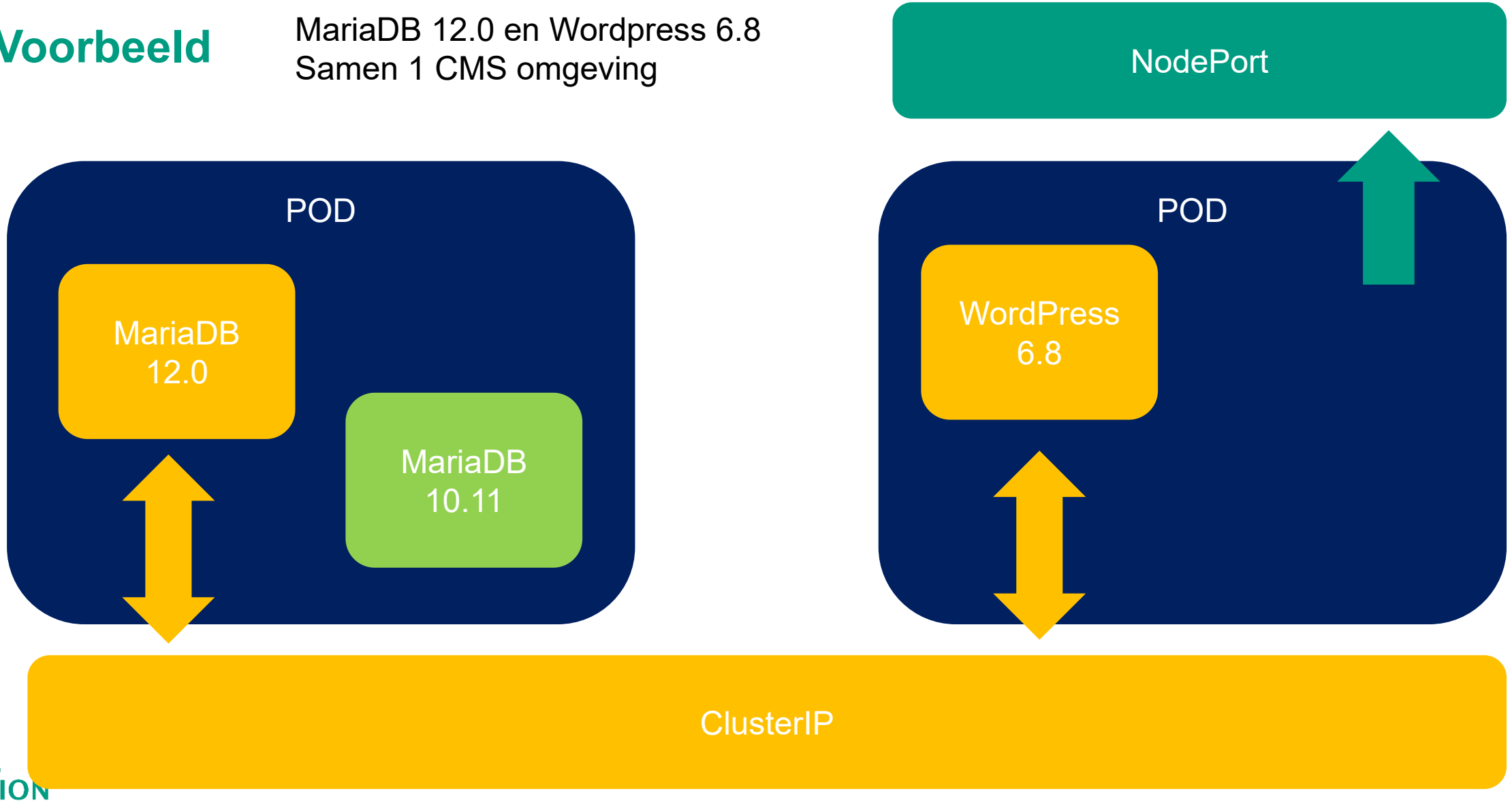



- NodePort
  - Exposes the Service on the same port of each selected Node in the cluster using NAT.
  - Makes a Service accessible from outside the cluster using <NodeIP>:<NodePort>. Superset of ClusterIP.
- Bron:
  https://kubernetes.io/docs/tutorials/kubernetes-basics/expose/expose-intro/

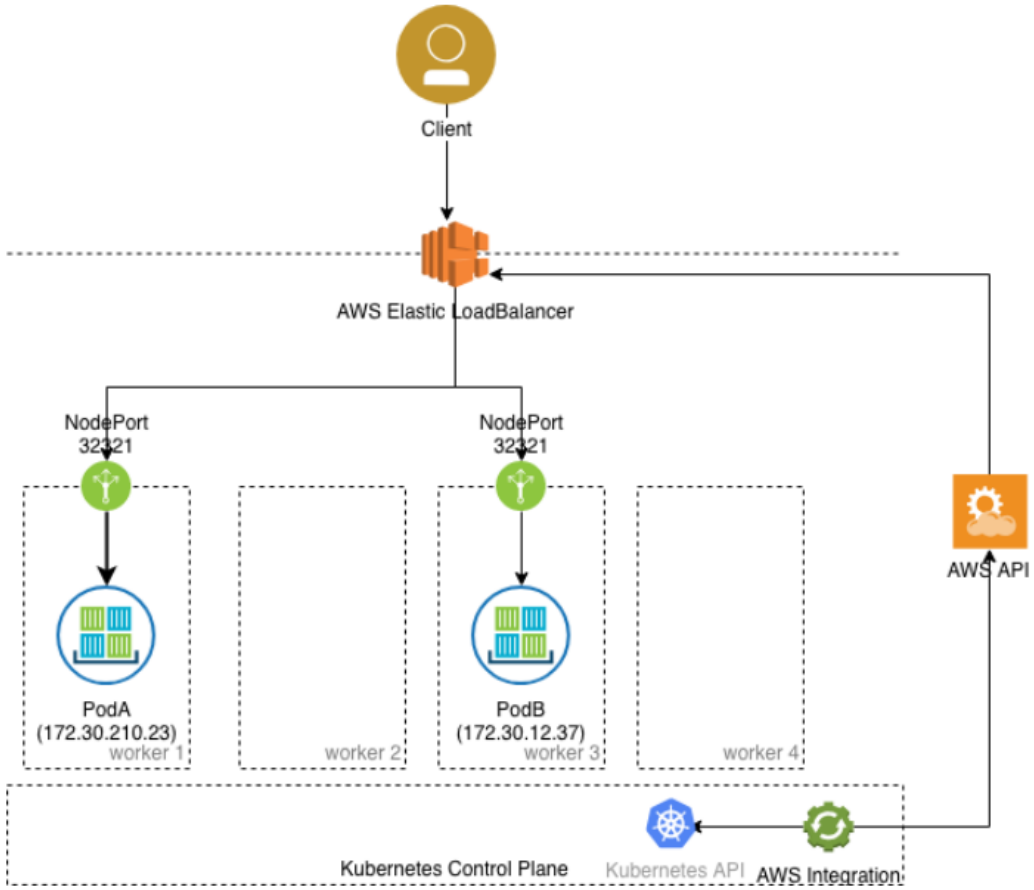
https://kubernetes.io/docs/concepts/services-networking/service/#type-nodeport

## Voorbeeld

MariaDB 12.0 en Wordpress 6.8
Samen 1 CMS omgeving



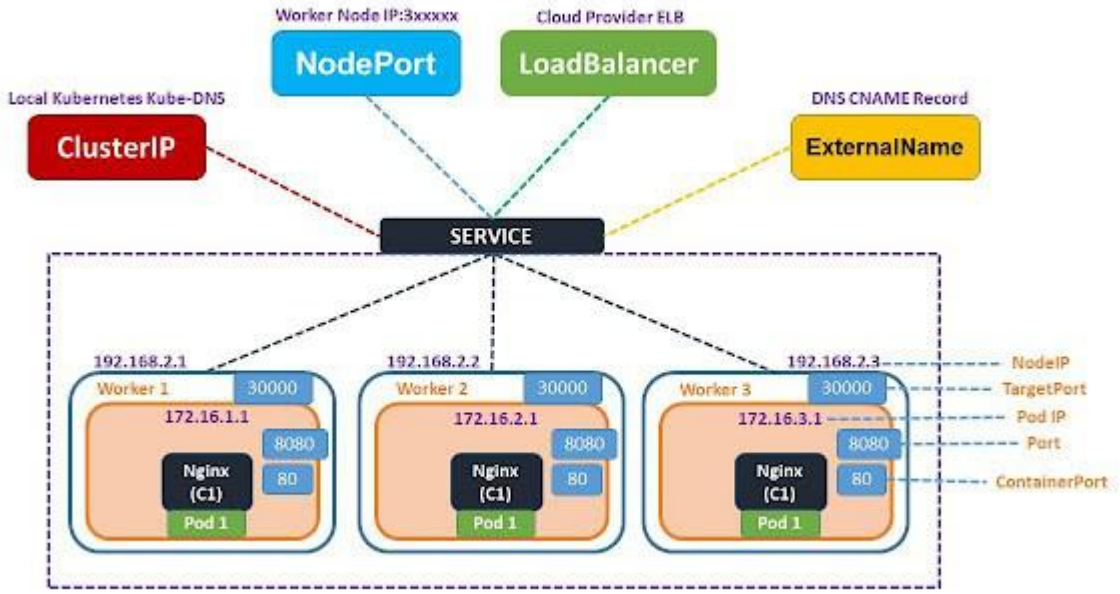
# Kubernetes | Deployment | Service | Exposing Services | LoadBalancer




- LoadBalancer
  - Exposes the Service externally using an external load balancer.
  - Creates an external load balancer in the current cloud (if supported) and assigns a fixed, external IP to the Service.
  - Superset of NodePort.
- Bron:
  https://kubernetes.io/docs/tutorials/kubernetes-basics/expose/expose-intro/

https://kubernetes.io/docs/concepts/services-networking/service/#loadbalancer

# Kubernetes | Deployment | Service | Exposing Services | ExternalName



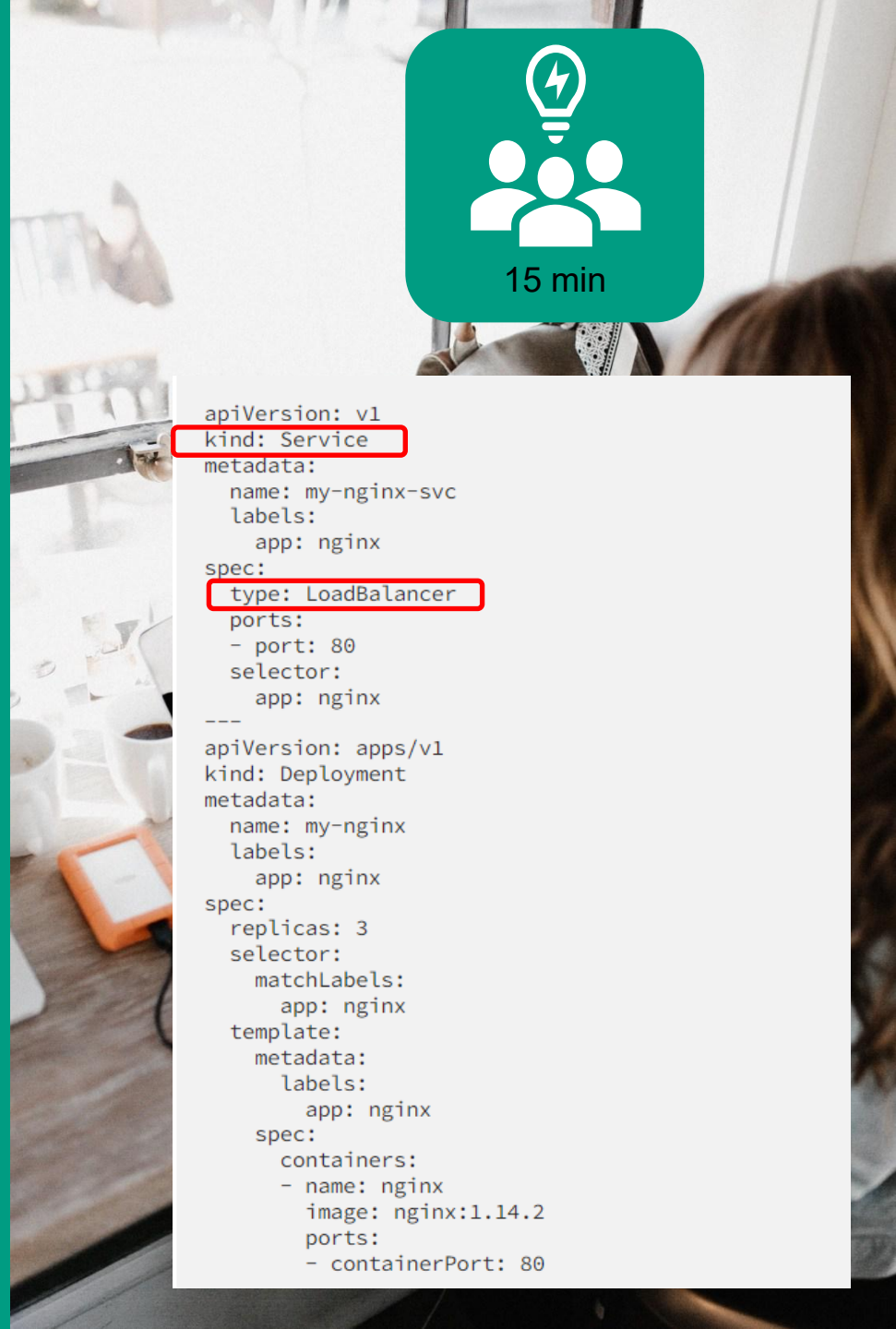

ExternalName

- Maps the Service to the contents of the externalName field (e.g. foo.bar.example.com), by returning a CNAME record with its value.
- No proxying of any kind is set up.
- This type requires v1.7 or higher of kube-dns, or CoreDNS version 0.0.8 or higher.

https://kubernetes.io/docs/concepts/services-networking/service/#externalname

# Opdracht

- Start eigen Kubernetes omgeving.
  - Docker Desktop
  - Docker Desktop met Minikube
  - Play with Kubernetes website (inlog GIT account)
- Geef commando:
  - kubectl *apply* -f https://raw.githubusercontent.com/kubernetes/website/master/content/en/examples/application/nginx-app.yaml




SAXION HOGESCHOOL

# Storage



kubernetes

12 min.

SAXION HOGESCHOOL

# Kubernetes | Storage | Opties |

- **HostPath:**
  - A hostPath volume mounts a file or directory from the host node's filesystem into your Pod
  - Met HostPath kan een container data opslaan op de host machine zelf. Deze optie is eenvoudig te configureren, maar heeft als nadeel dat de data niet gedeeld kan worden tussen verschillende nodes en er geen automatische replicatie is.
- **EmptyDir:**
  - Een EmptyDir is een tijdelijke volume die wordt gecreëerd wanneer een pod wordt gestart en verwijderd wanneer de pod eindigt. Dit is handig voor het tijdelijk opslaan van data die alleen nodig is tijdens de levensduur van de pod.



https://kubernetes.io/docs/concepts/storage/volumes/

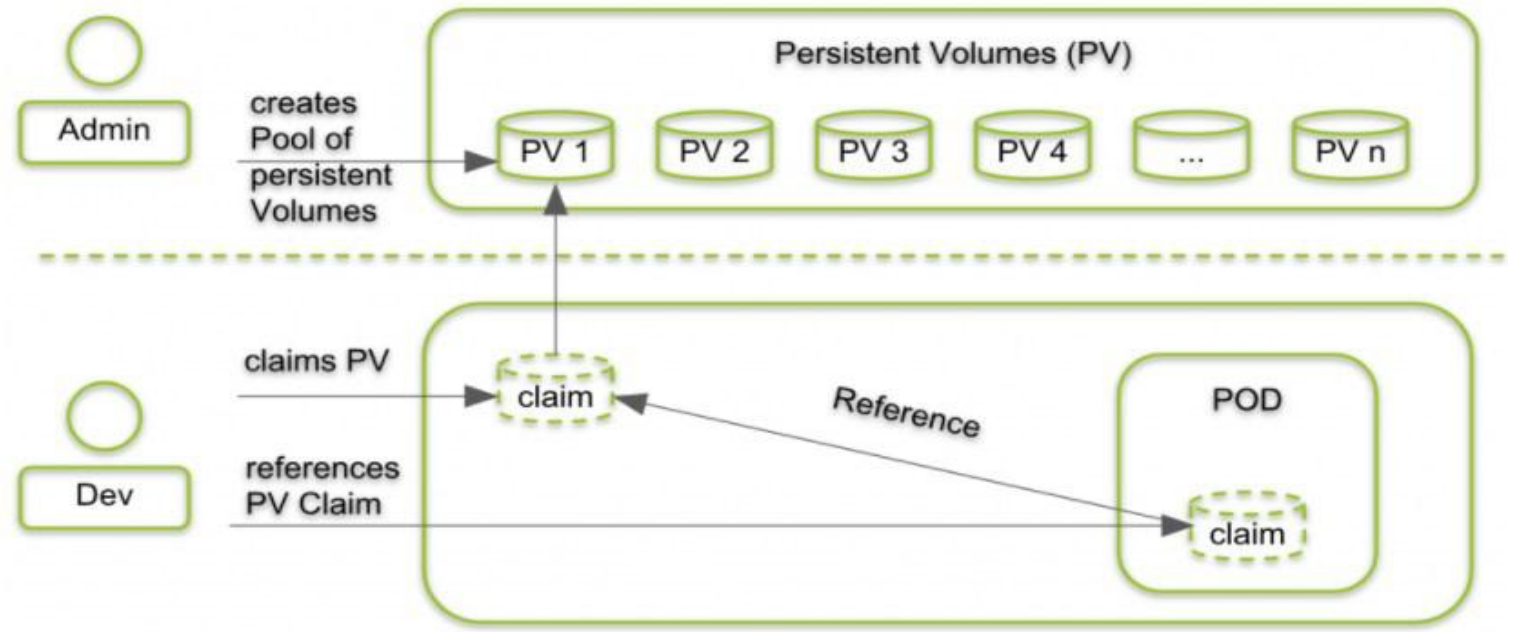
# Kubernetes | Storage | Opties |

- **PersistentVolume (PV) en PersistentVolumeClaim (PVC):**
  - PV's zijn onafhankelijke volumes die buiten de pod worden gedefinieerd en worden toegewezen aan PVC’s.
  - PVC's zijn aanvragen voor opslag die aan pods worden toegewezen op basis van de grootte en de opslagklasse.
  - Dit biedt flexibiliteit omdat het mogelijk is om verschillende typen opslag te gebruiken, zoals NFS, iSCSI, AWS EBS of Azure Disk.
- **StorageClass:**
  - Een StorageClass definieert de parameters voor dynamische provisioning van PV’s.
  - Het biedt de mogelijkheid om opslag op maat te maken voor verschillende workloads en om te profiteren van cloud-native opslagoplossingen zoals AWS EBS, Azure Disk of Google Persistent Disk.
- **StatefulSet:**
  - StatefulSets zijn bedoeld voor applicaties die een persistente identiteit vereisen.
  - Elke pod binnen een StatefulSet krijgt een unieke hostname en netwerkidentiteit, waardoor het gemakkelijker wordt om opslagvolumes te beheren die aan specifieke pods zijn gekoppeld.



https://kubernetes.io/docs/concepts/storage/persistent-volumes/

# PV, PVC




Persistent Volume (PV): Storage volume bijv. 10 GB op local disk in /mnt/data

Persistant Volume Claim (PVC) is wat er gevraagd wordt: bv tenminste 8 GB.
Als er een PV is dit voldoet wordt de PVC gekoppeld (bind).

Een pod kan vervolgens de PVC gebruiken (mounten).

# [1] Persistent Volume (PV)

Zie https://kubernetes.io/docs/reference/kubernetes-api/config-and-storage-resources/persistent-volume-v1/

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: task-pv-volume
  labels:
    type: local
spec:
  storageClassName: manual
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: "/mnt/data"
```

Creeert een persistent Volume *task-pv-volume* wat voldoet aan gevraagde requirements.

Type: local ➔ gebruik local storage
andere opties: nfs, iscsi, glusterfs, azurefile etc

StorageClassName: manual.
Hierin kunnen aanvullende eisen beschreven zijn.

Storage: gevraagde capaciteit

accessModes: ReadWriteOnce
slechts 1 pod mag tegelijk connecten

path: /mnt/data
pad wat gemount wordt

# [2] Persistent Volume Claim (PVC)

Zie https://kubernetes.io/docs/reference/kubernetes-api/config-and-storage-resources/persistent-volume-claim-v1/

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: task-pv-claim
spec:
  storageClassName: manual
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 3Gi
```

Er wordt nu storage gevraagd.

PVC *task-pv-claim* wordt gevraagd

Moet voldoen aan deze storageClass

Deze accessMode

En tenminste deze size

Volume *task-pv-volume* (vorige slide) voldoet dus.

VORIGE SLIDE

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: task-pv-volume
  labels:
    type: local
spec:
  storageClassName: manual
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: "/mnt/data"
```

# [3] Pod met persistent volume claim (PVC) storage

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: task-pv-pod
spec:
  volumes:
    - name: task-pv-storage
      persistentVolumeClaim:
        claimName: task-pv-claim
  containers:
    - name: task-pv-container
      image: nginx
      ports:
        - containerPort: 80
          name: "http-server"
      volumeMounts:
        - mountPath: "/usr/share/nginx/html"
          name: task-pv-storage
```

Volume krijgt de naam *task-pv-storage*

Nu wordt Pod gecreeerd die de toegekende PVC *task-pv-claim* (vorige slide) gaat gebruiken.

Volume *task-pv-storage* wordt in Pod gemount in dir /usr/share/nqinx/html

# Kubernetes | Opdracht



40 min

SAXION HOGESCHOOL

# Opdracht







40 min

- Ga bezig met bouwen omgeving voor Toepassen

- Gebruik voor YAML:
  - https://k8syaml.com/
  - https://8gwifi.org/kube.jsp
  - https://gimlet.io/k8s-yaml-generator
  - https://syshunt.com/kubernetes-yaml-generator/

Toepassen (10%)

- Je kan een webapplicatie actief binnen meerdere Pods in een cluster laten zien;
- Je kan laten zien dat een Workload/ReplicaSet gebruikt wordt voor deze applicatie door een Pod in een cluster te stoppen, waarna deze automatisch herstart;
- Je kan een applicatie updaten in een Pod binnen een cluster m.b.v. een Workload/Deployment en deze update aantonen;
- Je kan een Service gebruiken om een applicatie in een Pod binnen een cluster extern beschikbaar te maken. Dit demonstreer je door de applicatie te benaderen met een browser vanaf je host machine;
- Je kan persistente storage gebruiken in een applicatie. Dit demonstreer je door in de applicatie een wijziging aan te brengen, de applicatie te herstarten en te laten zien dat de wijziging niet verdwenen is.

# Huiswerk

- Ga naar Brightspace
- Ga naar Les 6.2
- Ga naar Verwerking
- Doe opdrachten



